



Software defect prediction using a bidirectional LSTM network combined with oversampling techniques

Nasraldeen Alnor Adam Khleel¹ · Károly Nehéz¹

Received: 10 May 2023 / Revised: 28 September 2023 / Accepted: 5 October 2023 / Published online: 28 October 2023
© The Author(s) 2023

Abstract

Software defects are a critical issue in software development that can lead to system failures and cause significant financial losses. Predicting software defects is a vital aspect of ensuring software quality. This can significantly impact both saving time and reducing the overall cost of software testing. During the software defect prediction (SDP) process, automated tools attempt to predict defects in the source codes based on software metrics. Several SDP models have been proposed to identify and prevent defects before they occur. In recent years, recurrent neural network (RNN) techniques have gained attention for their ability to handle sequential data and learn complex patterns. Still, these techniques are not always suitable for predicting software defects due to the problem of imbalanced data. To deal with this problem, this study aims to combine a bidirectional long short-term memory (Bi-LSTM) network with oversampling techniques. To establish the effectiveness and efficiency of the proposed model, the experiments have been conducted on benchmark datasets obtained from the PROMISE repository. The experimental results have been compared and evaluated in terms of accuracy, precision, recall, f-measure, Matthew's correlation coefficient (MCC), the area under the ROC curve (AUC), the area under the precision-recall curve (AUCPR) and mean square error (MSE). The average accuracy of the proposed model on the original and balanced datasets (using random oversampling and SMOTE) was 88%, 94%, And 92%, respectively. The results showed that the proposed Bi-LSTM on the balanced datasets (using random oversampling and SMOTE) improves the average accuracy by 6 and 4% compared to the original datasets. The average F-measure of the proposed model on the original and balanced datasets (using random oversampling and SMOTE) were 51%, 94%, And 92%, respectively. The results showed that the proposed Bi-LSTM on the balanced datasets (using random oversampling and SMOTE) improves the average F-measure by 43 and 41% compared to the original datasets. The experimental results demonstrated that combining the Bi-LSTM network with oversampling techniques positively affects defect prediction performance in datasets with imbalanced class distributions.

Keywords Software defect prediction · Software metrics · Artificial neural networks · Recurrent neural networks · Bidirectional long short-term memory · Class imbalance · Oversampling techniques

1 Introduction

Software defect prediction (SDP) is a critical component of software quality assurance, primarily aiming to detect potential defects early in the software development life cycle [1, 2]. There are various activities throughout the software development process to identify source code defects, including design reviews, code inspections, unit testing, integration testing, and other functions [3, 4]. Since software products must be free of defects to maintain customer satisfaction, identifying existing software defects is a primary concern in software engineering [5, 6]. To address this issue, SDP leverages tools or models such as

Nasraldeen Alnor Adam Khleel and Károly Nehéz have contributed equally to this work.

✉ Nasraldeen Alnor Adam Khleel
nasr.alnor@uni-miskolc.hu

Károly Nehéz
aitnehez@uni-miskolc.hu

¹ Department of Information Engineering, University of Miskolc, Miskolc 3515, Hungary

machine learning (ML) to predict source code defects based on historical data [7–9]. The SDP process depends on three main components: dependent variables, independent variables, and a model. Dependent variables are the defect data for the piece of code (defective or non-defective), which can be binary or ordinal variables. Independent variables (inputs) are the metrics that score the software code. The model contains the rules or algorithms which predict the dependent variable from the independent variables. The inputs (variables) are split into test and training data sets to determine the classifier's effectiveness. The training data set is used to create the classifier. Then it is used to predict potential defects in the test data set and evaluate these predictions using different performance measures to determine whether they are correct [10]. Software metrics have essential roles in SDP, and most SDP strategies rely on software metrics as independent variables. Object-oriented metrics have been designed to support finding faults in software projects. Due to the enormous variety of software applications, identifying, locating, and detecting software defects is becoming daunting for researchers. Furthermore, defect density is also challenging in software defect detection and prediction. Usually, defective software databases naturally consist of imbalanced data, which generates randomness in pattern characteristics; this motivates the development of an efficient and precise model for SDP [11].

Several methods have been proposed for SDP, but there is still a need to develop accurate defect detection models or detectors and robust software metrics to distinguish between defective and non-defective software modules. ML and data mining techniques are among the most promising approaches in software engineering, especially in SDP [12]. Various ML techniques have been applied to SDP, including traditional classifiers such as decision trees, support vector machines, and logistic regression. However, these methods have limitations when modeling sequential data, which is often the case in SDP [13]. RNNs have shown great promise in handling sequential data and have been applied to various applications, including natural language processing, speech recognition, and time-series prediction. In recent years, RNNs have gained significant attention in SDP due to their ability to capture temporal dependencies and patterns in software data [14]. Using RNNs in SDP involves training a neural network to learn from a dataset of software metrics, such as lines of code, code complexity, and code churn. The RNN model takes these metrics as inputs and learns to predict whether a software module will likely contain defects. The output of the RNN model can be binary (i.e., defect present or not) or a probability estimate of defect likelihood [15]. Several variations of RNNs, such as Long Short-Term Memory (LSTM) networks, Bidirectional Long Short-Term

Memory (Bi-LSTM), and Gated Recurrent Units (GRUs), have been applied in SDP to address the issue of vanishing gradients and improve performance. Bi-LSTM is a specialized variant of the Long Short-Term Memory (LSTM) recurrent neural network architecture. Bi-LSTM networks have demonstrated exceptional prowess in various sequence-related tasks, such as natural language processing and speech recognition. Their ability to capture temporal dependencies makes them a promising candidate for SDP, as the manifestation of defects often follows patterns that unfold over time [11].

However, while the capabilities of Bi-LSTM networks are substantial, they face challenges when applied to imbalanced datasets—a common scenario in SDP, where the number of non-defective instances dwarfs the number of defective ones. In pursuit of more accurate and balanced predictions, researchers and practitioners have turned to sampling techniques, notably oversampling techniques. These techniques aim to alleviate class imbalance by either replicating existing instances or generating synthetic samples. Oversampling techniques stand out as the preferred choice among data balancing methods to address class imbalance in the domain of SDP for several compelling reasons. Software defect occurrences are often infrequent and intricately patterned, necessitating a method that can capture these nuances effectively. Oversampling achieves this by generating additional instances of the minority class, ensuring that the model comprehends the subtle defect indicators. In the context of software engineering, obtaining large volumes of real-world defect data is challenging and resource intensive. Oversampling optimally leverages the available data, enhancing model learning without the need for exhaustive data collection efforts. Moreover, these techniques align well with the priority of achieving high recall rates in defect identification, making them a pragmatic solution for cost-effectively improving prediction accuracy [10, 15].

The uneven distribution of classes in the training data set indicates an imbalanced dataset. Imbalanced class classification biases performance towards the majority numbered class in the case of a binary application [16]. Most RNN techniques can predict better when each class's instances are roughly equal. Imbalanced classes severely hinder these models' efficiency and produce unbalanced false-positive and false-negative results [10, 16]. This study selects imbalanced datasets from the public PROMISE repository for experimental purposes [10, 15, 17]. The rationale behind selecting imbalanced datasets lies in our aim to investigate the efficacy of various sampling techniques and their potential to augment the accuracy of RNN models. By deliberately working with imbalanced datasets, we can closely examine the impact of methods such as oversampling in rectifying class imbalances. This empirical

exploration not only sheds light on the adaptability of RNN models to real-world scenarios where imbalanced data is prevalent but also provides insights into how these techniques can effectively tackle bias in predictive modelling, ultimately bolstering the reliability and generalizability of our results. Several experiments in the previous studies [2, 7, 9, 18] were conducted based on these datasets using many RNN models; most of the outcomes exhibited significant underperformance, primarily attributed to the studies not implementing any techniques to address the challenge of class imbalance. However, to our knowledge, there is no experiment using a Bi-LSTM Network combined with oversampling techniques in the literature.

To bridge these gaps, this study delves into the innovative realm of defect prediction by harnessing the power of Bi-LSTM Networks, a cutting-edge deep learning technique, coupled with the strategic application of oversampling techniques (Random Oversampling and Synthetic Minority Oversampling Technique “SMOTE”). By combining these two powerful approaches, we aim to address the longstanding challenge of imbalanced datasets in software defect prediction. This innovative approach leverages the temporal understanding of the Bi-LSTM Network to capture intricate defect patterns and augments it with oversampling techniques to effectively address class imbalance issues. Our motivation lies in the potential to revolutionize defect prediction accuracy, enabling proactive identification and mitigation of software defects. By exploring this fusion of machine learning and software engineering, we aspire to provide software practitioners with a new toolset that empowers them to create more reliable and resilient software systems. Firstly, we apply oversampling techniques to balance the training set. Secondly, we train and test the proposed Bi-LSTM model using the balanced training set, and finally, we evaluate the results based on many performance measures. In summary, the goal and main contributions of our study are summarized as follows:

- (i) In this study, we propose a novel model that combines a Bi-LSTM network with oversampling techniques to predict software defects.
- (ii) We evaluate the proposed model’s performance and compare it with the traditional ML model (RF) as the baseline and existing models used in SDP.
- (iii) We show that the performance of the Bi-LSTM network in SDP can be significantly improved when balancing the data set by applying oversampling techniques.

The structure of this paper is organized as follows. Section 2 presents a discussion on related work. Section 3 presents background on the topics of RNNs and Bi-LSTM Network. Section 4 presents the hypothesis and research

questions. After that, our research methodology is presented in Sect. 5. Section 6 presents the experimental results and discussion. Section 7 presents the implications of the findings. Section 8 presents threats to validity, followed by conclusions in the last section (Sect. 9).

2 Related work

The prediction of defects in software systems is significant, and there is great interest in developing novel high-performance software defect predictors. The purpose of SDP models is to improve the quality of software. Many models have been constructed to recognize the defects in software modules using artificial intelligence and statistical methods. RNN [2, 9, 18–21], Support Vector Machine [3, 22], ANNs [23], K-Nearest Neighbors (KNN) [24] and Deep Neural Networks (DNN) [12, 25] are some of the algorithms used for SDP.

Ayon [2] proposed a method based on different neural network models. The models were evaluated based on five different datasets from NASA using different scales. The experimental results showed that the proposed method is suitable for predicting software defects. This method used different performance measures and achieved high prediction accuracy. Kumar and Satyanarayana [6] developed a Hybrid Neural Network model with object-oriented and CK metrics for software fault prediction. Adaptive Genetic Algorithm has been used for ANN optimization. The proposed model has been tested with PROMISE data sets. The experimental results showed better performance compared to major existing schemes. Miholca et al. [9] presented a supervised classification approach named (HyGRAR). It is a nonlinear hybrid model that combines gradual relational association rule mining and ANNs to predict software defects. Experiments were conducted using ten open-source datasets; their results showed excellent performance of the proposed classifier and better performance than most previously proposed classifiers. Their method achieved high prediction accuracy. Khleel and Nehéz [10] presented a model based on a convolutional neural network (CNN) and gated recurrent unit (GRU) combined with a synthetic minority oversampling technique plus the Tomek link (SMOTE Tomek) to predict software defects. The historical data obtained from the PROMISE repository were used to evaluate the experiments. The experimental results have been compared and evaluated using several performance measures. The experimental results demonstrate that the proposed models perform better and that there are positive effects of combining CNN and GRU models with the SMOTE Tomek method on the performance of SDP regarding datasets with imbalanced class distributions, and the proposed approach is a more promising alternative for

addressing the problem of class imbalance in SDP as compared with previous methods. Arar and Ayan [13] proposed a hybrid classifier to predict software defect problems. The performance of the proposed classifier was compared with other algorithms based on five datasets, and the results show its performance is better. The method used different performance measures and achieved high prediction accuracy. Deng et al. [15] proposed a novel LSTM method to perform SDP; their method can automatically learn semantic and contextual information from the program's ASTs. The experiment was performed on several open-source projects, showing that the proposed LSTM method is superior to the state-of-the-art methods. Khleel and Nehéz [16] presented a model based on a combination of two recurrent neural networks (RNNs), namely long-short-term memory (LSTM) and gated recurrent unit (GRU), along with an undersampling method (near miss) to predict software bugs. The historical data obtained from the GitHub repository were used to evaluate the experiments. The experimental results have been compared and evaluated using several performance measures. The experimental results lead to the conclusion that the proposed models outperform others and the combination of RNN models with undersampling methods leads to improved bug prediction performance, particularly for datasets with imbalanced class distributions. Ye et al. [18] proposed a classification model using an LSTM network to classify bugs based on 9000 bug reports from three software projects. The results of the evaluation and comparison showed that their model achieves the best results. Farid et al. [19] proposed a hybrid model using Bi-LSTM and a convolutional neural network (CNN) to predict software defects. The proposed model was evaluated using seven open-source Java projects from the PROMISE dataset. Their results showed that the proposed model is accurate for predicting software defects. Zhou and Lu [20] developed an LSTM network based on bidirectional and tree structure (LSTM-BT) to predict software defects based on eight pairs of Java open-source projects. The evaluation results showed that the proposed model performs better than several state-of-the-art defect prediction models. Samir et al. [25] proposed a new method using DNN to predict software defects. Their method has been compared with some ML algorithms. Experimental results showed that the proposed method slightly improved over the other methods. This method achieved high prediction accuracy. Alsaeedi and Khan [26] proposed a study to compare the most well-known ML algorithms widely used to predict software defects. The performances of models were evaluated based on other performance metrics. The SMOTE resampling strategy was used to mitigate the data imbalance issues. Evaluation results showed that some of the proposed models performed well. Damet et al. [27]

developed a novel prediction LSTM model, which can automatically learn features for representing source code and using them for SDP. The model was evaluated based on two datasets, one from open-source projects contributed by Samsung and the other from the public PROMISE repository. The experimental results showed the effectiveness of the proposed model for both within-project and cross-project predictions. Pandey et al. [28] proposed a new method using deep representation and ensemble learning with sampling techniques for software bug prediction and dealing with the class imbalance problem. The experiment was performed based on 12 data sets from the PROMISE repository. Evaluation results showed that the proposed method outperformed other state-of-the-art techniques. This method solved the class imbalance problem. Fan et al. [29] presented an SDP framework via an attention based RNN. The models were evaluated based on an open-source Apache Java project, using F1-measure and area under the curve (AUC). Experimental results demonstrated that the proposed model improves the F1 measure by 14% and AUC by 7% compared with the state-of-the-art methods. Khuat and Le [30] presented an empirical study regarding the importance of combining sampling techniques and ML models on unbalanced data in SDP. The experimental results indicated the positive effects of combining sampling techniques and ML models on defect prediction performance concerning data sets with unbalanced class distributions. This method solved the class imbalance problem. Amirabbas Majd et al. [31] proposed SLDeep using LSTM as a learning model, a technique for statement-level SDP based on more than 100,000 C/C++ projects. The evaluation results showed that the proposed model seems effective at statement-level SDP and can be adopted. This method used different performance measures and achieved high prediction accuracy. Bani-Salameh et al. [32] proposed a framework using LSTM for automatically assigning bugs. The proposed model has been validated on five real projects. The performance of the model was compared with two ML algorithms. The results showed that LSTM predicts and assigns the priority of the bug more accurately and effectively. Feng et al. [33] investigated the role of SMOTE-based and stable SMOTE-based oversampling techniques in improving SDP. The approach was evaluated based on four common classifiers across 26 datasets from the PROMISE Repository. The experimental analysis showed that the performance of stable SMOTE-based oversampling techniques is more stable and better than that of SMOTE-based oversampling techniques. This method solved the class imbalance problem. Liang et al. [34] proposed Seml, a novel framework that combines word embedding and LSTM for software defect prediction. The model was evaluated based on eight open-source projects. The experimental results showed that the Seml outperforms

three state-of-the-art defect prediction approaches on most datasets for both within-project and cross-project defect prediction.

After reviewing previous studies in SDP, we noticed that most proposed methods ignore the class imbalance problem. According to studies that dealt with the issue of class imbalance and addressed it, the authors point out that the data balancing methods have an essential role in improving SDP accuracy [10, 16, 26, 28, 30, 33]. So, the primary point from the recent studies is that ML combined with data balancing methods can improve and increase prediction accuracy. Therefore, our study focuses on addressing the class imbalance problem using oversampling techniques.

3 Recurrent neural networks (RNNs) and Bi-LSTM Networks

Recurrent Neural Networks (RNNs) are ANNs that can process a sequence of inputs and retain their state while processing the following sequence of inputs and efficiently acquiring the nonlinear features that are in order. The nodes and their connections form a temporally directed graph along a temporal sequence [2]. RNN is widely used to solve many problems, such as pattern recognition, identification, classification, vision, speech, control systems, etc. [25]. Due to the problem of long-term dependencies that arise when the input sequence is too long, RNN cannot guarantee a long-term nonlinear relationship. This means the learning sequence has a gradient vanishing and gradient explosion phenomenon. Many optimization theories and improved algorithms have been introduced to solve this problem. To name a few, LSTM networks, Bidirectional RNNs, gated recurrent unit networks, and independent RNNs [35]. RNNs can use memory units (internal state) to learn the relationship between the sequence pieces, making it possible for RNNs to capture contextual features of the sequence [29].

Long-Short Term Memory (LSTM) Networks are a particular type of RNN used in deep learning, designed to recognize patterns in data sequences. LSTM Networks were introduced to avoid or handle long-term dependency problems without being affected by an unstable gradient. This problem frequently occurs in regular RNNs when connecting previous information to new information [18, 33]. A standard LSTM unit comprises a cell, an input gate, an output gate, and a forget gate. The cell remembers values over arbitrary time intervals, and the three gates regulate the flow of information into and out of the cell. Due to the ability of the LSTM network to recognize longer sequences of time-series data, LSTM models can provide high predictive performance in SDP [29, 35]. More

recently, Bi-LSTM networks are a new way to train data by expanding the capabilities of LSTM networks; it uses two separate hidden layers to train the input data twice in the forward and backward directions, as shown in Fig. 1. With the regular LSTM Networks, the input flows in one direction, either backward or forward. Bi-LSTM Networks are the process of making any neural networks have the sequence information in both directions (a sequence processing model that consists of two LSTM): one taking the input in a forward direction (past to future), and the other in a backward direction (future to past) [29, 32, 33]. We build a Bi-LSTM Network because the defective source code closely relates to its previous and subsequent source code segments. The idea behind Bi-LSTM Networks is to exploit spatial features to capture bidirectional temporal dependencies from historical data to overcome the limitations of traditional RNNs. Standard RNNs take sequences as inputs; each sequence step refers to a specific moment. For a particular moment t , the output o_t not only depends on the current input x_t but is also influenced by the output from the previous moment $t - 1$. The output of moment (t) can be formulated as the following equations:

$$h_t = f(U \times x_t + W \times h_{t-1} + b) \quad (1)$$

$$o_t = g(V \times h_t + c)$$

where U , V , and W denote the weights of the RNN, b , and c represent the bias, and f and g are the activation functions of the neurons. The cell state carries the information from the previous moments and will flow through the entire LSTM chain, which is the key that LSTM can have long-should be filtered from the previous moment; the output of the forget gate can be formulated as the following equation:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (2)$$

where σ denotes the activation function, W_f and b_f Denote the weights and bias of the forget gate, respectively. The input gate determines what information should be kept from the current moment, and its output can be formulated as the following equation:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (3)$$

where σ denotes the activation function, W_i and b_i Denote the weights and bias of the input gate, respectively. With the information from the forgetting gate and input gate, the cell state C_{t-1} is updated through the following formula:

$$\hat{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (4)$$

$$\hat{C}_t = f_t \times C_{t-1} + i \times \hat{C}_t$$

$\hat{C}_t$ Is a candidate value that is going to be added to the cell state and C_t Is the current updated cell state. Finally, the output gate decides what information should be

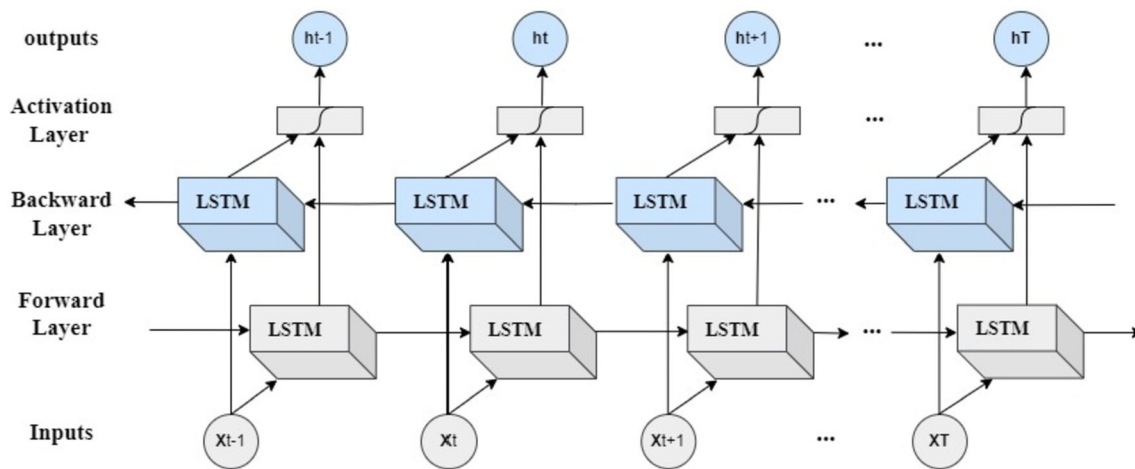


Fig. 1 Interacting layers of the repeating module in a Bi-LSTM “Figure adapted from Yegesh Verma [36]”

outputted according to the previous output and current cell state.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t + b_o] \quad (5)$$

$$h_t = o_t \times \tanh(C_t).$$

4 Hypothesis and research questions

Our hypothesis in this study is if data balancing techniques are applied to balance the original data sets, the classification performance of the proposed model will be better in SDP. To investigate our hypothesis, we used a paired t-test to determine whether there was a statistically significant difference between our model on the original and balanced datasets based on accuracy. The formula for the paired t-test is shown in Eq. 6 below. To statistically prove the validity of the impact of data balancing techniques on the performance of the Bi-LSTM model, the hypothesis is formed as follows:

H1: There is no difference in the model’s accuracy when there are no data balancing techniques and when the data balancing techniques are used.

H2: There is a difference in model accuracy when there are no data balancing techniques and when data balancing techniques are used.

$$t = \frac{m}{s/\sqrt{n}} \quad (6)$$

where m is the mean differences, n is the sample size (i.e., number of pairs), and s is the standard deviation.

Based on our hypothesis, this study aims to understand the impact of data-balancing techniques on the performance of the Bi-LSTM model in SDP. In particular, we aim to address the following research questions.

RQ1: Do data balancing techniques improve the accuracy of the Bi-LSTM model in SDP?

This RQ aims to investigate data balancing techniques to improve the Bi-LSTM model’s accuracy in SDP.

RQ2: Does the proposed Bi-LSTM model outperform the state-of-the-art models in SDP?

This RQ aims to investigate the performance of the proposed Bi-LSTM model in SDP compared against the state-of-the-art models.

5 Methodology

This study uses several software metrics to help build an SDP model that will outperform other SDP models. The experiments were performed on public benchmark datasets. The proposed method aims to improve the performance of software defect prediction by effectively addressing imbalanced data and capturing complex patterns in the data using a Bidirectional LSTM network. A series of steps have been taken and described, such as benchmark datasets used, software metrics used, data pre-processing, features selection, dataset balancing, model building, and evaluation. Figure 2 illustrates the whole workflow of the proposed method of SDP, where each step of the workflow is described in the following sections.

5.1 Benchmark datasets

To verify the validity of the proposed method, we selected six open-source Java projects from the PROMISE dataset [37]. All six projects’ source codes and corresponding PROMISE data are public [10, 15, 19, 38]. These projects cover applications such as XML parsers, text search engine libraries, and data transport adapters, and these projects have traditional static metrics for each Java file. To

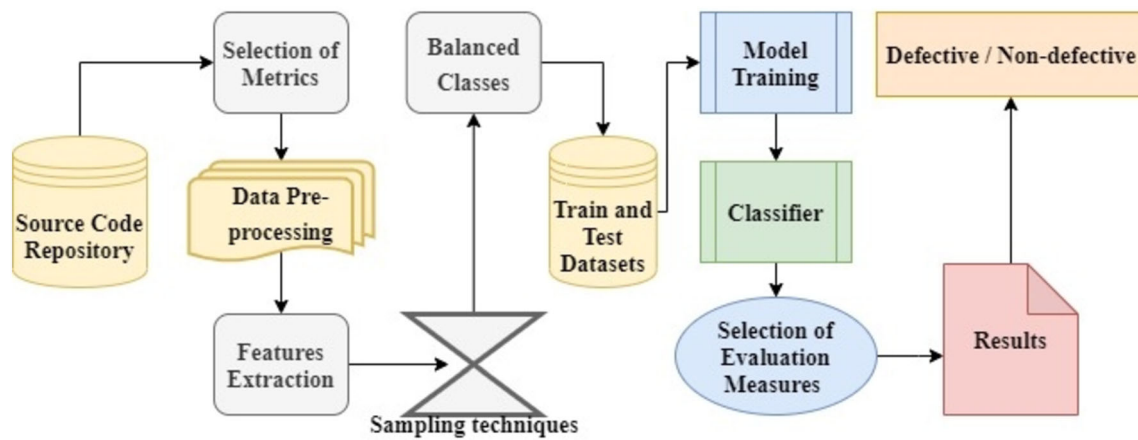


Fig. 2 Proposed method of SDP

guarantee the generality of the evaluation results, experimental datasets consist of projects with different sizes and defect rates (in the six projects, the maximum number of instances is 965, and the minimum number of instances is 205. In addition, the minimum defect rate is 2.23% and the maximum defect rate is 92.19%). Table 1 shows the essential information of selected projects, including project name, project version, number of instances, and defect rate or the percentage of defective instances.

5.2 Software metrics

Metrics are essential in software products for quality assurance, performance, debugging, and management. It also has a vital role in discovering defects in software components. Different software metrics are used for defect prediction [21, 23]. The metrics used to predict software defects are crucial in building a prediction model to improve software quality. Metrics can be divided into code metrics and process metrics. Code metrics indicate the complexity of the source code, while process metrics indicate the complexity of the development process. Code metrics are directly collected from source code and are mainly used to measure the source code's properties, e.g., size and complexity, etc.

Further, source code of higher complexity can be more defect prone. Process metrics are extracted from historical information archived in software repositories. These metrics reflect the modifications over time, e.g., changes in source code, the number of code changes, developer information, etc. [6]. Several researchers in the primary studies used McCabe and Halstead metrics as independent variables in SDP. The first use of McCabe metrics was to characterize code features related to software quality. McCabe's has considered four basic software metrics: cyclomatic complexity, essential complexity, design complexity, and lines of code. Halstead also considered that the

Table 1 Description of the PROMISE datasets that we have chosen

Project name	Project version	# Of instances	Defect rate %
ant	1.7	745	22.28
camel	1.6	965	19.48
ivy	2.0	352	11.36
jedit	4.3	492	2.23
log4j	1.2	205	92.19
xerces	1.4	588	74.31

software metrics fall into three groups: base measures, derived measures, and line of code measures [34, 39]. Table 2 shows the traditional static code metrics contained in the PROMISE repository, and for the descriptions, the readers are referred to [38].

5.3 Data pre-processing and features selection

Pre-processing the collected data is one of the essential stages before constructing the model. To generate a good model, data quality needs to be considered [11, 24]. Not all data collected is suitable for training and model building. The inputs will significantly impact the model's performance and later affect the output. Data pre-processing is a group of techniques applied to the data to remove noise and unwanted outliers from the data set, deal with missing values, feature type conversion, etc., to improve data quality before building the model [17, 19]. Normalization is necessary to convert the values into scaled values (scaling the data in numeric variables from 0 to 1) to increase the model's efficiency. Therefore, the data set was normalized using Min–Max normalization. The formula for calculating the normalized score can be described in (7). Feature selection is crucial in selecting the most

Table 2 List of 20 traditional static metrics of PROMISE

Attribute	Description
dit	The maximum distance from a given class to the root of an inheritance tree
noc	Number of children of a given class in an inheritance tree
cbo	Number of classes that are coupled to a given class
rfe	Number of distinct methods invoked by code in a given class
lcom	Number of method pairs in a class that do not share access to any class attributes
lcom3	Another the lcom metric proposed by Henderson–Sellers
npm	Number of public methods in a given class
loc	Number of lines of code in a given class
dam	The ratio of the number of private/protected attributes to the total number of attributes in a given class
moa	Number of attributes in a given class that are of user-defined types
mfa	Number of methods inherited by a given class divided by the total number of methods that can be accessed by the member methods of the given class
cam	The ratio of the sum of the number of different parameter types of every method in a given class to the product of the number of methods in the given class and the number of different method parameter types in the whole class
ic	Number of parent classes that a given class is coupled to
cbm	Total number of new or overwritten methods that all inherited methods in a given class are coupled to
amc	The average size of methods in a given class
ca	Afferent coupling, which measures the number of classes that depend on a given class
ce	Efferent coupling, which measures the number of classes that a given class depends on
max_cc	The maximum McCabe’s cyclomatic complexity (CC) score of methods in a given class
avg_cc	The arithmetic mean of McCabe’s cyclomatic complexity (CC) scores of methods in a given class

Descriptions were given in [38]

discriminative features using appropriate feature selection methods [40]. The goal of feature selection is to choose the features more relevant to the target class from high-dimensional features and remove the redundant and uncorrelated features [1, 41]. Feature extraction facilitates the conversion of pre-processed data into a form that the classification engine can use [42]. Feature selection methods are categorized into three categories (i) Filter methods: these methods are model agnostic, i.e., variables are selected independently of ML algorithms. These methods are faster and less computationally expensive, (ii) Wrapper methods: these methods are greedy and choose the best feature subsets in each iteration according to ML algorithms. It is a continuous process of finding a feature subset. These methods are very computationally expensive and often unrealistic if the feature space is vast, (iii) Embedded methods: in these methods, feature selection is a part of building ML algorithms. Embedded feature selection methods are a category of techniques used in ML and data analysis, where the process of feature selection is incorporated directly into the model training process. Unlike traditional feature selection methods that are performed before model training, embedded methods select

relevant features as the model learns from the data. These methods select the best possible feature subset per the ML model to be implemented [43]. In this study, our model relied on embedded methods because these methods fit ML models (faster and less computationally expensive than other methods).

$$x_i = (x_i - Xmin)/(Xmax - Xmin) \quad (7)$$

where $max(x)$ and $min(x)$ represent the maximum and minimum value of the attribute x , respectively.

5.4 Class imbalance and sampling techniques

Class imbalance is one of the significant challenges of ML models. It represents the cases where the number of examples of one class is much smaller than others [44]. Class imbalance is an essential special of the software defects data, which consists of only a few defective and many non-defective instances. Hence, the class imbalance problem often could cause misclassification of the instances in the minority class. Many standard methods are used to deal with the issue of class imbalance, such as bagging and boosting-based ensemble methods, cost-sensitive

learning techniques, sampling techniques, etc. [10, 16, 30]. The datasets used in our study suffer from a common problem in SDP studies: class imbalance [10, 15, 17]. The reference datasets are not correctly distributed, which is a lack in the actual distribution of learning instances; as shown in Table 1, we manage this problem by modifying the original datasets to increase the realism of the data. The distribution of the dataset is modified by applying sampling techniques. Sampling techniques used to deal with imbalanced class distributions might be divided into oversampling and under-sampling. Oversampling techniques supplement instances of the minority class to the dataset, while the under-sampling techniques eliminate samples of the majority class to obtain a balanced dataset. In SDP, oversampling techniques are favored for handling class imbalance. They adeptly capture complex defect patterns that arise from rare events with subtle indicators. By generating extra minority class instances, oversampling prevents information loss and overfitting. With scarce defect data and its associated costs, oversampling optimizes existing data, enhances recall rates, and seamlessly integrates into algorithms, providing an efficient solution for predictive accuracy [19]. In this study, to deal with class imbalance, we use oversampling techniques [Random oversampling and Synthetic Minority Oversampling Technique (SMOTE)]. Random oversampling is a method that involves randomly selecting examples from the minority class with replacements and adding them to the training dataset [30]. SMOTE is a method in which new samples of minority class are synthesized based on the feature space similarities among existing minority examples [22, 33]. The distribution of learning defective instances over the original data sets (ant, camel, ivy, jedit, log4j, and xerces) is (166, 188, 40, 11, 16, and 151), respectively. While the distribution of learning non-defective instances is (579, 777, 312, 481, 189, and 437), respectively. Following the implementation of oversampling techniques (Random oversampling and SMOTE), the distribution of learning defective instances over the balanced data sets (ant, camel, ivy, jedit, log4j, and xerces) became (579, 777, 312, 481, 189, 437), respectively. While

the distribution of learning non-defective instances became (579, 777, 312, 481, 189, and 437), respectively. Figure 3 shows the distribution of learning instances over the original and balanced data sets.

5.5 Model building and evaluation

In previous works, many RNN algorithms have been developed for SDP. Most studies of SDP divide the data into two sets: a training set and a test set. The training set is used to train the model, whereas the testing set is used to evaluate the performance of the defect's prediction model. Once a defects prediction model is built, its performance must be evaluated. Implementation framework of our models: we use Keras as a high-level API based on TensorFlow to build our models for simplicity and correctness; training is performed with 80% of the dataset (random selection of features), while the remaining 20% is used for testing and validation, each model was developed separately with different parameters as shown in Table 3. Models that predict software defects in binary classification problems are usually evaluated using a confusion matrix, MCC, AUC, AUCPR, and MSE as Loss functions.

A confusion matrix is a specific table used to measure the performance of a model. A confusion matrix summarizes the results of the testing algorithm and presents a report of (1) True Positive Rate (TPR), (2) False Positive Rate (FPR), (3) True Negative Rate (TNR), and (4) False Negative Rate (FNR). From the values in the confusion matrix, various performance metrics can be derived, such as accuracy, precision, recall, and F1-measure shown in the below equations. These metrics provide insights into the model's strengths and weaknesses, especially in scenarios where class imbalance is present. Table 4 shows the confusion matrix.

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{FP} + \text{FN} + \text{TN}) \quad (8)$$

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) \quad (9)$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN}) \quad (10)$$

$$F - \text{Measure} = (2 * \text{Recall} * \text{Precision}) / (\text{Recall} + \text{Precision}) \quad (11)$$

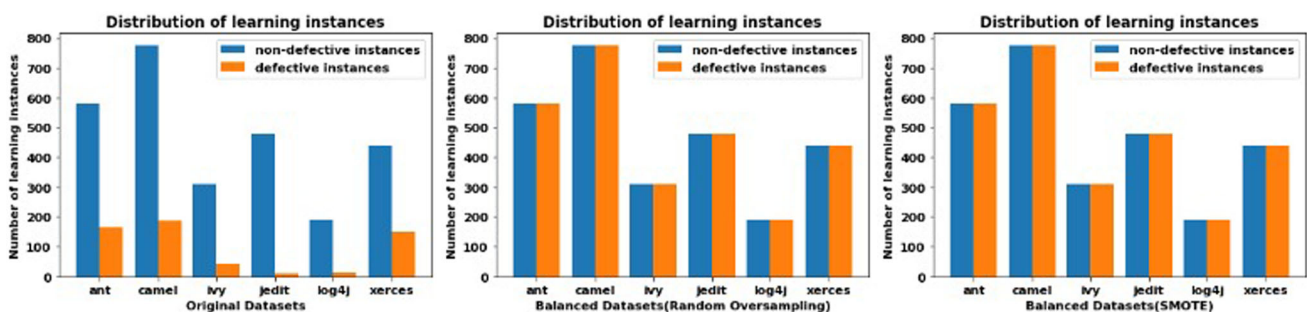


Fig. 3 Distribution of learning instances over the original and balanced data sets

Table 3 Parameters settings of the model

Bi-LSTM	Parameters
Cell type (Bidirectional)	LSTM (64, 32), return_sequences = True
Activation function	ReLU + sigmoid
Dropouts	0.2
Dense	64, 1
Optimizer	Adam
Learning_Rate	0.01
Loos Function	Mean squared error “MSE”
Batch_Size	64
Epochs	100
Validation_Split	0.1
Verbose	1

Table 4 Confusion matrix

Predicted	Actual	
	Defective	Non-defective
Defective	TN	FP
Non-defective	FN	TP

The MCC is a performance metric for binary classification. MCC is used for model evaluation by measuring the difference and describing the correlation between the predicted and actual values. The MCC formula is shown in the equation below:

$$\text{MCC} = \frac{\text{TP} * \text{TN} - \text{FP} * \text{FN}}{\sqrt{(\text{TP} + \text{FP}) * (\text{TP} + \text{FN}) * (\text{TN} + \text{FP}) * (\text{TN} + \text{FN})}} \quad (12)$$

AUC graph shows the performance of classification models with all classification thresholds and plots based on two parameters, actual positive rate (TPR.) and false-positive rate (FPR.). The AUC formula is shown in the equation below:

$$\text{AUC} = \frac{\sum_{ins_i \in \text{PositiveClass}} \text{rank}(ins_i) - \frac{M(M+1)}{2}}{M * N} \quad (13)$$

where $\sum_{ins_i \in \text{PositiveClass}} \text{rank}(ins_i)$ It is the sum of the ranks of all positive samples, and M and N are the numbers of positive and negative samples, respectively.

AUCPR is a curve that plots the Precision versus the Recall or a single number summary of the information in

the precision-recall curve. The AUCPR formula is shown in the equation below:

$$\text{AUCPR} = \int_0^1 \text{Precision}(\text{Recall})d(\text{Recall}) \quad (14)$$

MSE is a metric that measures the amount of error in the model. It assesses the average squared difference between the actual and predicted values. The MSE formula is shown in the equation below:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (x(i) - y(i))^2 \quad (15)$$

where n is the number of observations, $x(i)$ is the actual value, $y(i)$ is the observed or predicted value for the i^{th} observation.

6 Experimental results and discussion

In this section, we evaluate the efficiency of our proposed model. The experiment was performed in a Python environment. The study has considered six open-source datasets for experimental analysis using the Bi-LSTM Network. We also did experiments using the traditional ML model (RF) as a baseline model and compared it with our proposed Bi-LSTM model.

To answer the research question—RQ1, the prediction model’s performance is reported in Tables 5, 6, 7, 8, and Figs. 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 are mentioned below.

According to Table 5: Accuracy for the various original datasets: the highest accuracy was achieved by the proposed model on the jedit dataset, which is 97%. The lowest accuracy was achieved by the proposed model on the ant dataset, which is 80%. Precision for the various original datasets: the highest Precision was achieved by the proposed model on the log4j and xerces datasets, which is 95%. The proposed model achieved the lowest Precision on the jedit dataset, 0%. Recall for the various original datasets: the highest Recall was achieved by the proposed model on the log4j dataset, which is 100%. The lowest Recall was achieved by the proposed model on the jedit dataset, which is 0%. F-Measure for the various original datasets: the highest F-Measure was achieved by the proposed model on the log4j dataset, which is 97%. The lowest F-Measure was achieved by the proposed model on the jedit dataset, which is 0%. MCC for the various original datasets: the highest MCC was achieved by the proposed model on the xerces dataset, which is 75%. The lowest

Table 5 Performance analysis for proposed Bi-LSTM Network—original datasets

Datasets	Performance measures							
	Accuracy	Precision	Recall	F-Measure	MCC	AUC	AUCPR	MSE
ant	0.80	0.50	0.50	0.50	0.37	0.79	0.48	0.152
camel	0.82	0.56	0.28	0.37	0.30	0.69	0.37	0.146
ivy	0.87	0.50	0.22	0.31	0.27	0.72	0.40	0.105
jedit	0.97	0.00	0.00	0.00	0.00	0.85	0.29	0.030
log4j	0.95	0.95	1.00	0.97	0.00	0.60	0.96	0.041
xerces	0.91	0.95	0.92	0.94	0.75	0.94	0.98	0.075
Averages	0.88	0.57	0.48	0.51	0.28	0.76	0.58	0.091

Table 6 Performance analysis for proposed Bi-LSTM network—balanced datasets using random oversampling technique

Datasets	Performance measures							
	Accuracy	Precision	Recall	F-Measure	MCC	AUC	AUCPR	MSE
ant	0.91	0.89	0.94	0.91	0.82	0.95	0.93	0.073
camel	0.91	0.87	0.98	0.92	0.81	0.93	0.92	0.082
ivy	0.90	0.82	1.00	0.90	0.81	0.93	0.86	0.092
jedit	0.99	0.98	1.00	0.99	0.97	0.99	0.99	0.009
log4j	0.99	1.00	0.98	0.99	0.97	0.99	0.99	0.012
xerces	0.95	0.98	0.92	0.95	0.89	0.97	0.98	0.049
Averages	0.94	0.92	0.97	0.94	0.87	0.96	0.94	0.052

Table 7 Performance analysis for proposed Bi-LSTM network—balanced datasets using SMOTE technique

Datasets	Performance measures							
	Accuracy	Precision	Recall	F-Measure	MCC	AUC	AUCPR	MSE
ant	0.84	0.81	0.88	0.85	0.67	0.90	0.91	0.124
camel	0.87	0.89	0.88	0.89	0.74	0.91	0.91	0.113
ivy	0.89	0.83	0.97	0.89	0.78	0.94	0.92	0.101
jedit	0.99	0.98	1.00	0.99	0.97	0.99	0.99	0.011
log4j	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.001
xerces	0.93	0.93	0.92	0.93	0.85	0.96	0.97	0.067
Averages	0.92	0.90	0.94	0.92	0.83	0.95	0.95	0.069

Table 8 Comparison of the results of the proposed Bi-LSTM Model based on the original and balanced datasets in terms of accuracy using paired t-test

Paired t-test	Original datasets	Balanced datasets using random oversampling	Original datasets	Balanced datasets using SMOTE
Mean	0.88	0.94	0.88	0.92
STD	0.06	0.04	0.06	0.06
Min	0.80	0.90	0.80	0.84
Max	0.97	0.99	0.97	1.00
P value	0.01		0.00	

MCC was achieved by the proposed model on the jedit and log4j datasets, which is 0%. AUC for the various original datasets: the highest AUC was achieved by the proposed model on the xerces dataset, 94%. The lowest AUC was achieved by the proposed model on the log4j dataset,

which is 60%. AUCPR for the various original datasets: the highest AUCPR was achieved by the proposed model on the xerces dataset, 98%. The lowest AUCPR was achieved by the proposed model on the jedit dataset, which is 29%. MSE for the various original datasets: the highest MSE was

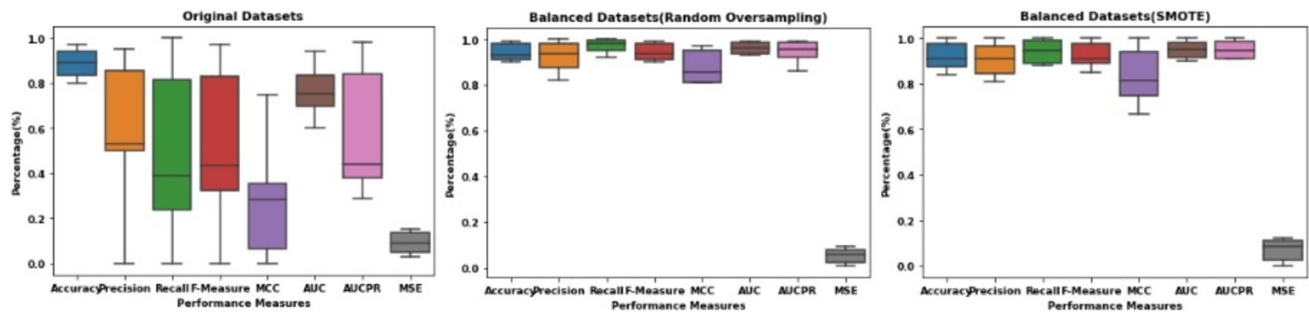


Fig. 4 Boxplots represent performance measures obtained by the model on the original and balanced datasets

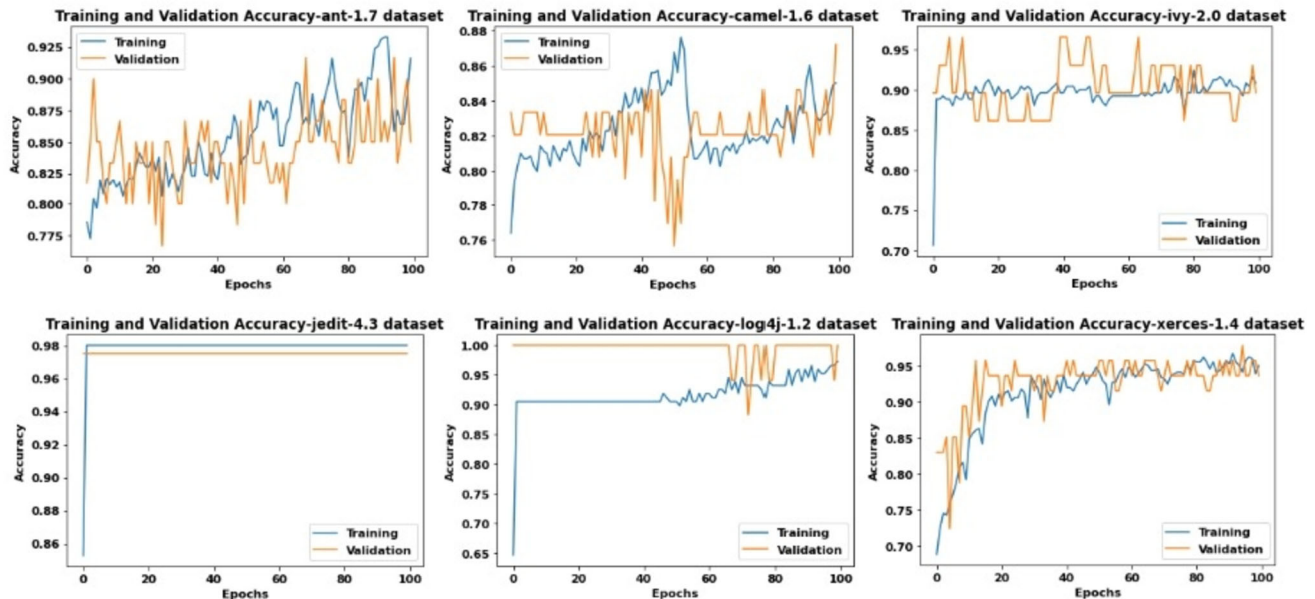


Fig. 5 Training and validation accuracy for the original datasets

achieved by the proposed model on the ant dataset, which is 0.152. The lowest MSE was achieved by the proposed model on the jedit dataset, which is 0.030.

According to Table 6: accuracy for the various balanced datasets using random oversampling: the highest accuracy was achieved by the proposed model on the jedit and log4j datasets, which is 99%. The lowest accuracy was achieved by the proposed model on the ivy dataset, which is 90%. Precision for the various balanced datasets using random oversampling: the highest Precision was achieved by the proposed model on the log4j dataset, which is 100%. The proposed model on the ivy dataset achieved the lowest Precision, which is 82%. Recall for the various balanced datasets using random oversampling: the highest Recall was achieved by the proposed model on the ivy and jedit datasets, which is 100%. The lowest Recall was achieved by the proposed model on the xerces dataset, which is 92%. F-Measure for the various balanced datasets using random oversampling: the highest F-Measure was achieved by the proposed model on the jedit and log4j datasets, which is

99%. The lowest F-Measure was achieved by the proposed model on the ivy dataset, which is 90%. MCC for the various the various balanced datasets using random oversampling: the highest MCC was achieved by the proposed model on the jedit and log4j datasets, which is 97%. The lowest MCC was achieved by the proposed model on the camel and ivy datasets, which is 81%. AUC for the various balanced datasets using random oversampling: the highest AUC was achieved by the proposed model on the jedit and log4j datasets, which is 99%. The lowest AUC was achieved by the proposed model on the camel and ivy datasets, which is 93%. AUCPR for the various balanced datasets using random oversampling: the highest AUCPR was achieved by the proposed model on the jedit and log4j datasets, which is 99%. The lowest AUCPR was achieved by the proposed model on the ivy dataset, which is 86%. MSE for the various balanced datasets using random oversampling: the highest MSE was achieved by the proposed model on the ivy dataset, which is 0.092. The lowest

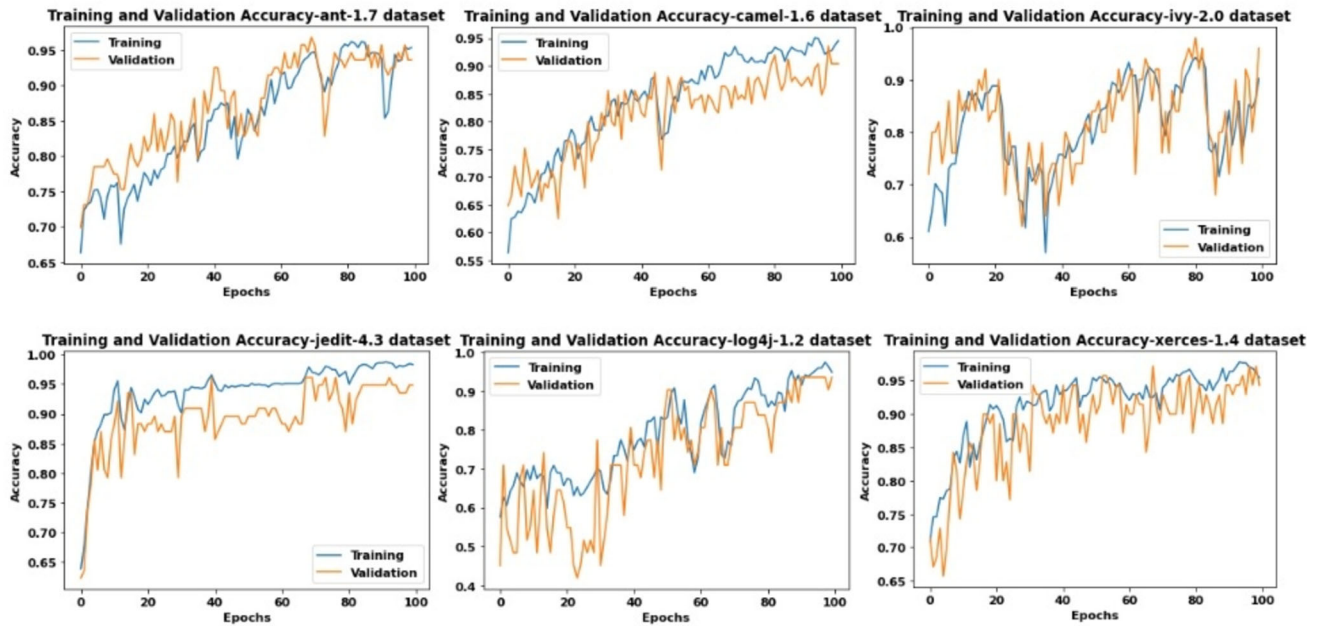


Fig. 6 Training and validation accuracy for the balanced datasets—random oversampling

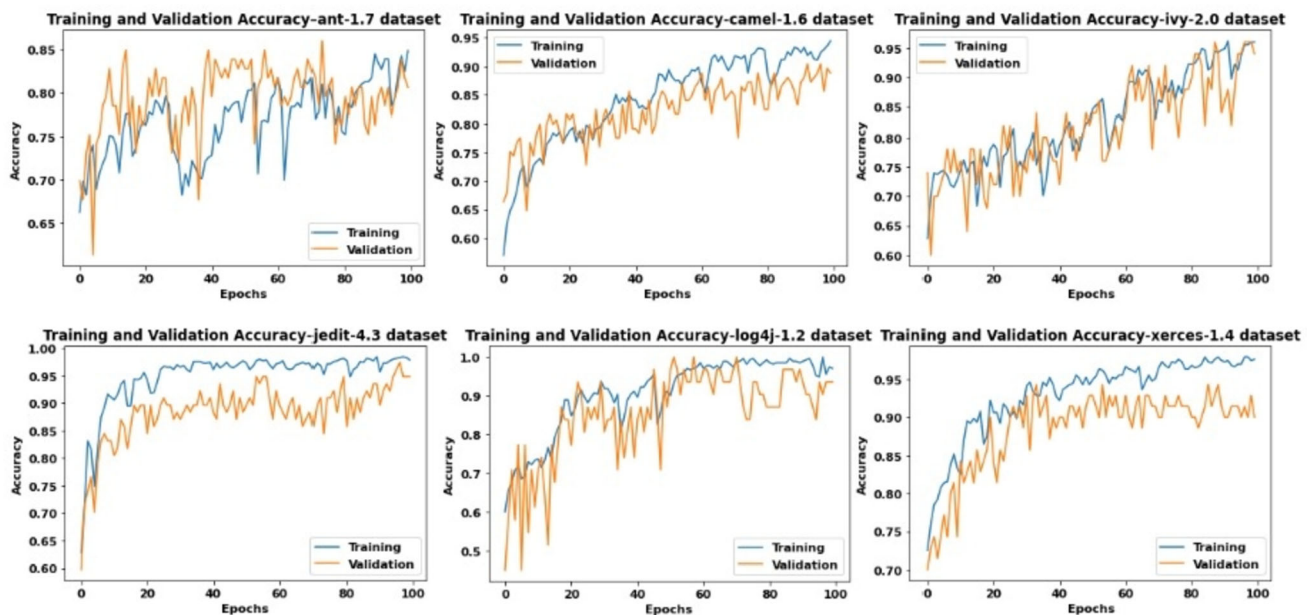


Fig. 7 Training and validation accuracy for the balanced datasets—SMOTE

MSE was achieved by the proposed model on the jedit dataset, which is 0.009.

According to Table 7: accuracy for the various balanced datasets using SMOTE: the highest accuracy was achieved by the proposed model on the log4j dataset, which is 100%. The proposed model achieved the lowest accuracy on the ant dataset, 84%. Precision for the various balanced datasets using SMOTE: the highest Precision was achieved by the proposed model on the log4j dataset, which is 100%.

The lowest Precision was achieved by the proposed model on the ant dataset, which is 81%. Recall for the various balanced datasets using SMOTE: the highest Recall was achieved by the proposed model on the jedit and log4j datasets, which is 100%. The lowest Recall was achieved by the proposed model on the ant and camel datasets, which is 88%. F-Measure for the various balanced datasets using SMOTE: the highest F-Measure was achieved by the proposed model on the log4j dataset, which is 100%. The

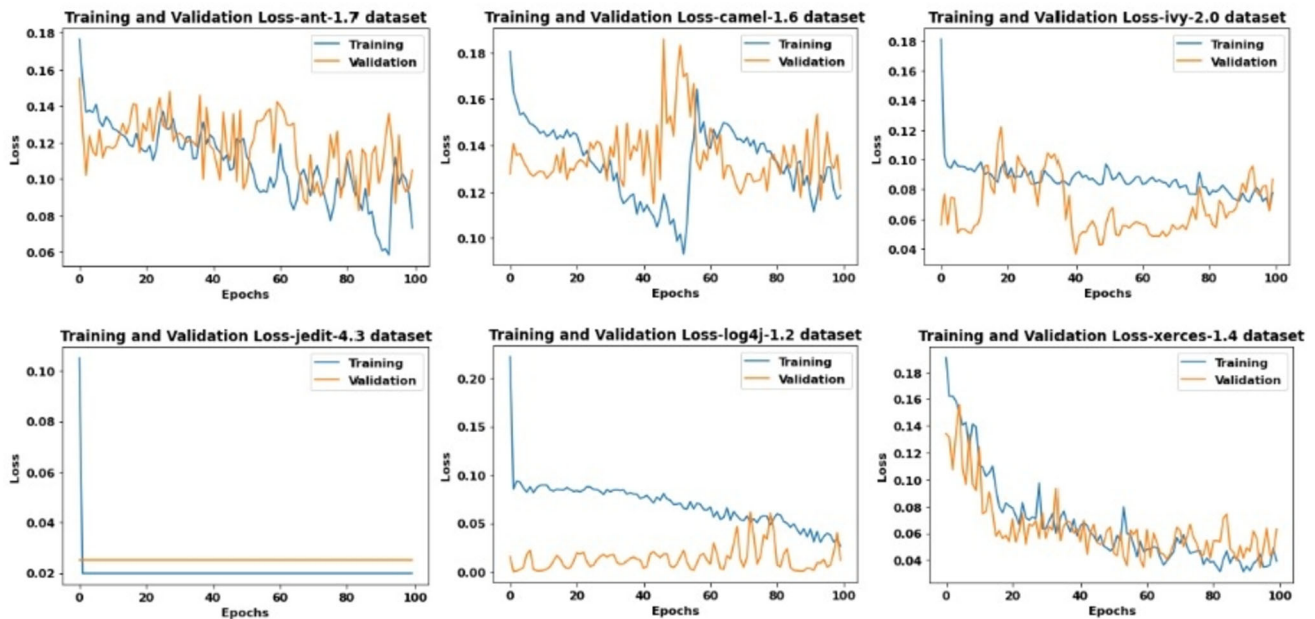


Fig. 8 Training and validation loss for the original datasets

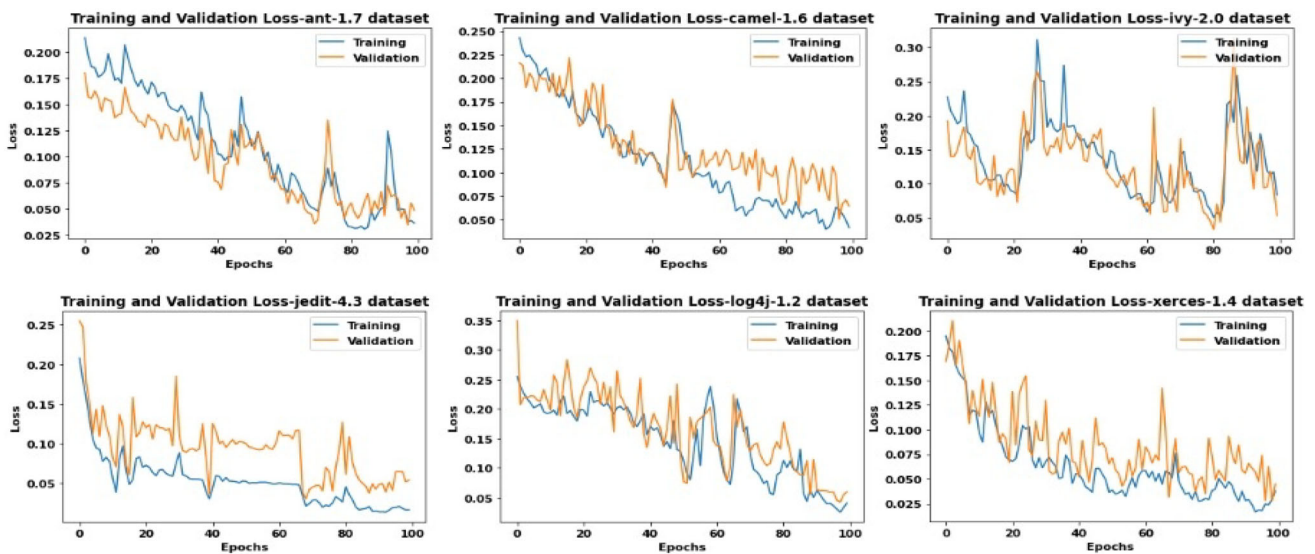


Fig. 9 Training and validation loss for the balanced datasets—random oversampling

lowest F-Measure was achieved by the proposed model on the ant dataset, which is 85%. MCC for the various balanced datasets using SMOTE: the highest MCC was achieved by the proposed model on the log4j dataset, which is 100%. The lowest MCC was achieved by the proposed model on the ant dataset, which is 67%. AUC for the various balanced datasets using SMOTE: the highest AUC was achieved by the proposed model on the log4j dataset, which is 100%. The lowest AUC was achieved by the proposed model on the ant dataset, which is 90%. AUCPR for the various balanced datasets using SMOTE: the highest AUCPR was achieved by the proposed model

on the log4j dataset, which is 100%. The lowest AUCPR was achieved by the proposed model on the ant and camel datasets, which is 91%. MSE for the various balanced datasets using SMOTE: the highest MSE was achieved by the proposed model on the ant dataset, which is 0.124. The lowest MSE was achieved by the proposed model on the log4j dataset, which is 0.001.

Table 8 presents the statistical analysis results (paired t-test) of the proposed model on the original and balanced datasets (using random oversampling and SMOTE) in terms of mean, Standard Deviation (STD), min, max, and P value. We notice that the mean values of the Bi-LSTM

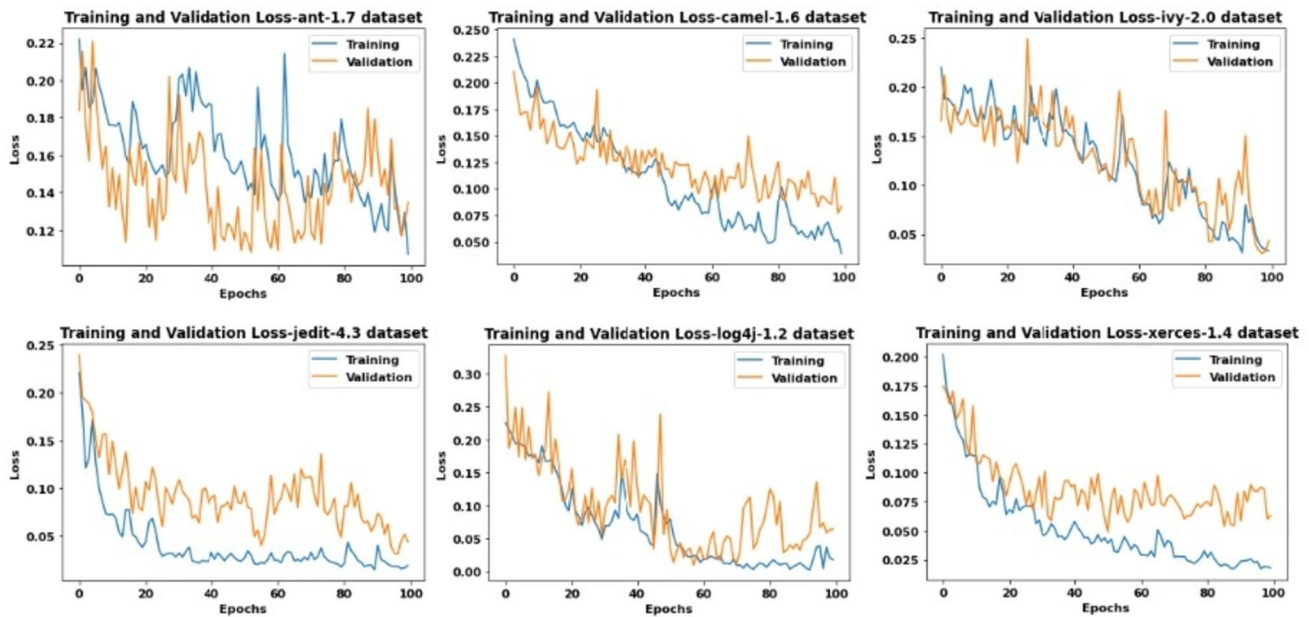


Fig. 10 Training and validation loss for the balanced datasets—SMOTE

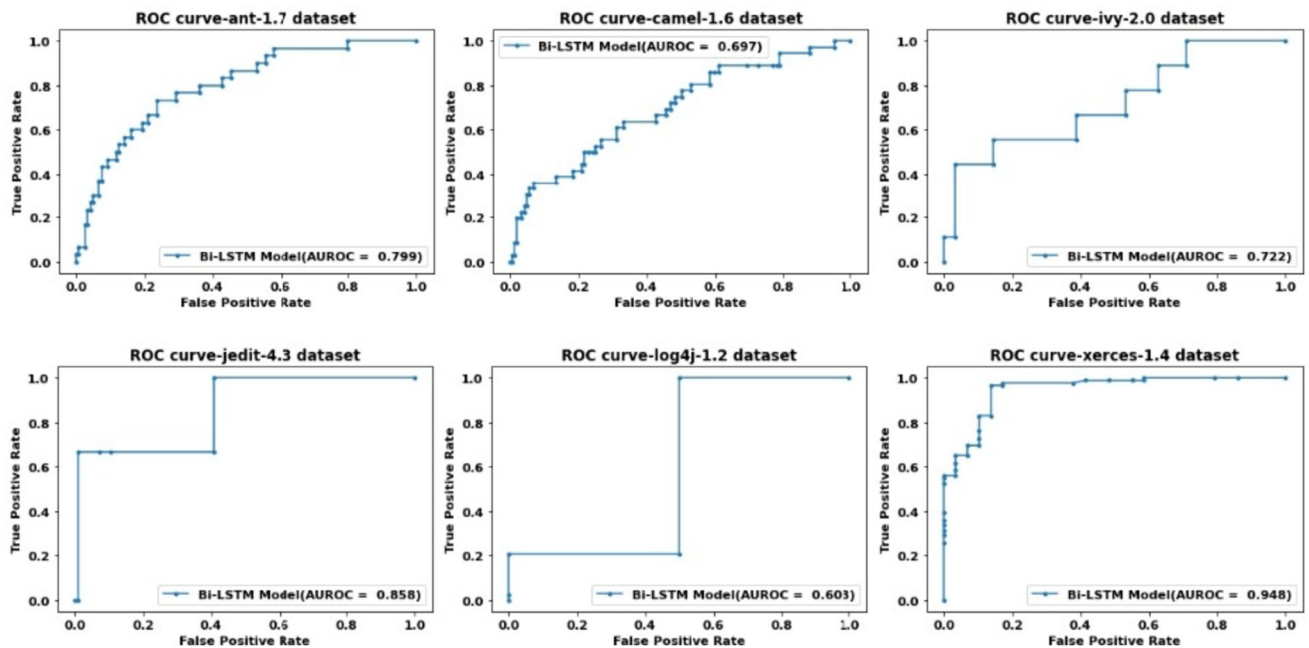


Fig. 11 ROC curves for the original datasets

model are 0.88 on the original datasets, 0.94 on the balanced datasets using random oversampling, and 0.92 on the balanced datasets using SMOTE. The STD values of the Bi-LSTM model are 0.06 on the original datasets, 0.04 on the balanced datasets using random oversampling, and 0.06 on the balanced datasets using SMOTE. The Min values of the Bi-LSTM model are 0.80 on the original datasets, 0.90 on the balanced datasets using random oversampling, and 0.84 on the balanced datasets using SMOTE. The Max

values of the Bi-LSTM model are 0.97 on the original datasets, 0.99 on the balanced datasets using random oversampling, and 1.00 on the balanced datasets using SMOTE. The P value of the Bi-LSTM model is 0.01 on the original and balanced datasets using random oversampling and 0.00 on the original and balanced datasets using SMOTE. Based on the P value of the model on the original and balanced data sets, we note that the P value is less than

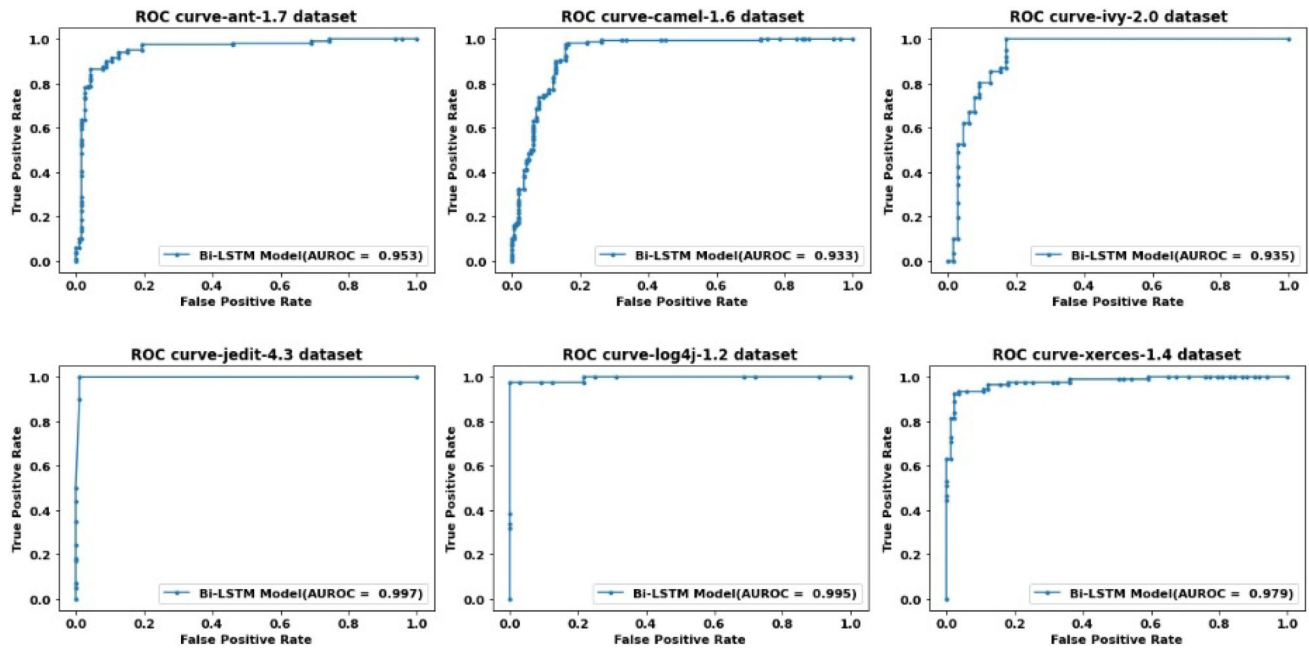


Fig. 12 ROC curves for the balanced datasets—random oversampling

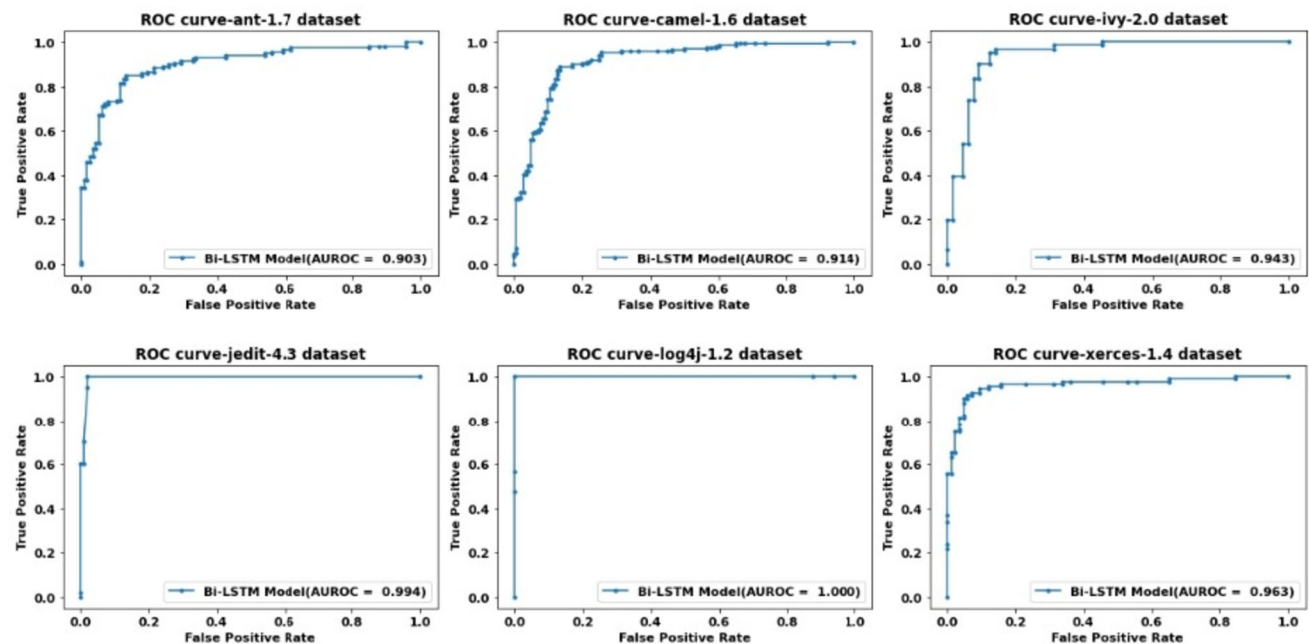


Fig. 13 ROC curves for the balanced datasets—SMOTE

0.05, which indicates a difference between the results of the model on the original and balanced data sets.

Boxplots are very useful for describing the distribution of results and providing raw results for comparing different techniques. Therefore, we aggregated the achieved results to get a more accurate overview of the quality of the results using boxplots. Figure 4 below shows the Box plots for the performance measures (Accuracy, Precision, Recall,

F-measure, MCC, AUC, AUCPR, and MSE) on the original and balanced datasets: The averages of (Accuracy, Precision, Recall, F-measure, MCC, AUC, AUCPR, and MSE) on the original datasets are 0.88, 0.57, 0.48, 0.51, 0.28, 0.76, 0.58, and 0.091, respectively. The averages of (Accuracy, Precision, Recall, F-measure, MCC, AUC, AUCPR, and MSE) on the balanced data sets (using random oversampling) are 0.94, 0.92, 0.97, 0.94, 0.87, 0.96,

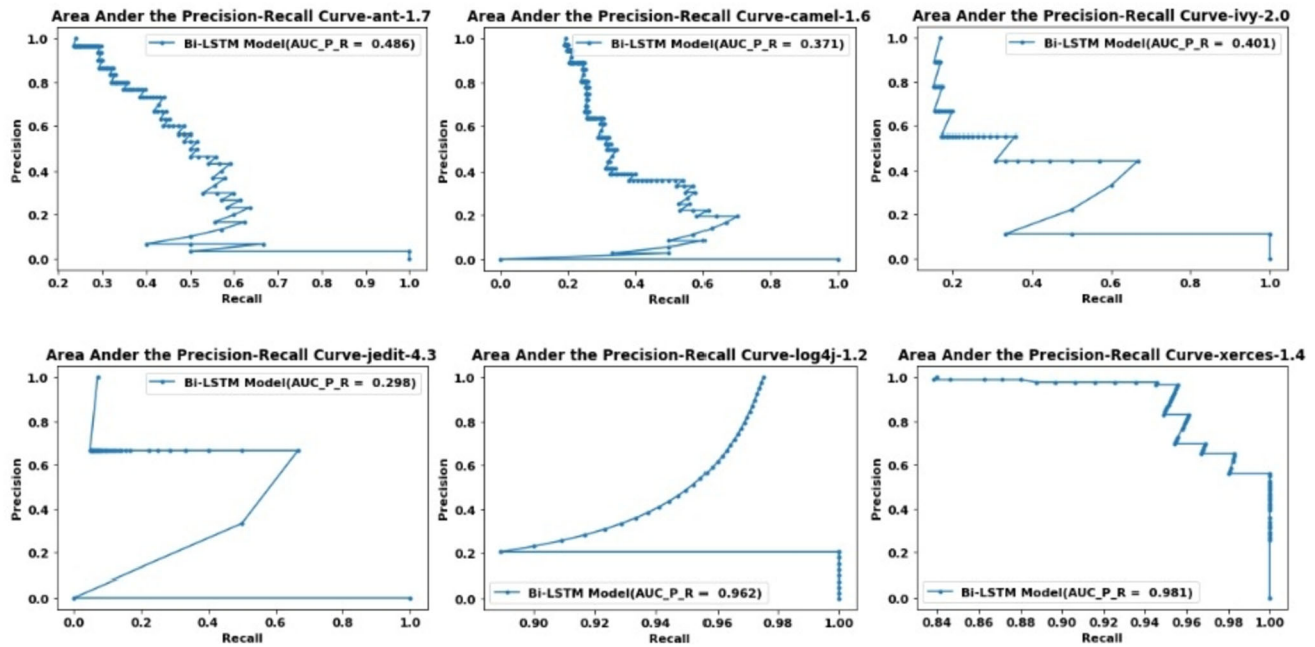


Fig. 14 AUCPR for the original datasets

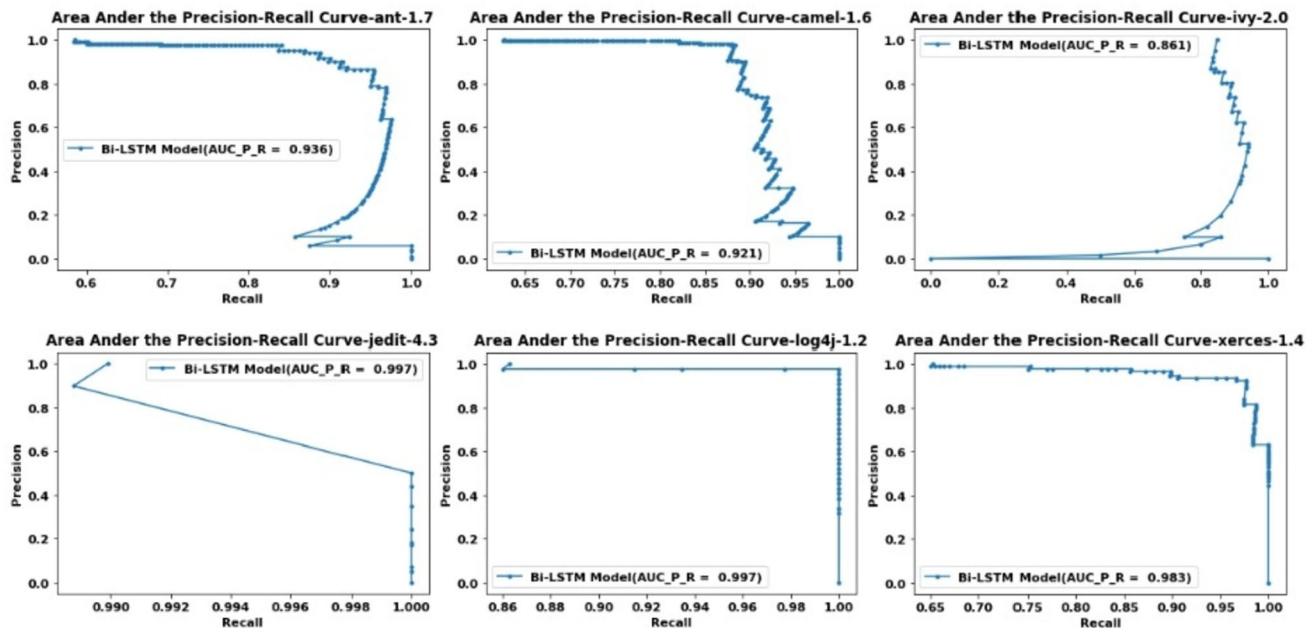


Fig. 15 AUCPR for the balanced datasets—random oversampling

0.94, and 0.052, respectively. The averages of (Accuracy, Precision, Recall, F-measure, MCC, AUC, AUCPR, and MSE) on the balanced data sets (using SMOTE) are 0.92, 0.90, 0.94, 0.92, 0.83, 0.95, 0.95, and 0.069, respectively.

Figures 5, 6, and 7 show the training and validation accuracy of the model on the original and balanced datasets. The vertical axis presents the accuracy of the model, and the horizontal axis illustrates the number of epochs.

Accuracy is the fraction of predictions that our model predicted right.

Figure 5 shows the accuracy values of the model on the original datasets. From the Figure, the model learned 80% accuracy for the ant dataset, 82% accuracy for the camel dataset, 87% accuracy for the ivy dataset, 97% accuracy for the jedit dataset, 95% accuracy for the log4j dataset, and 91% accuracy for xerxes dataset at the 100th epoch.

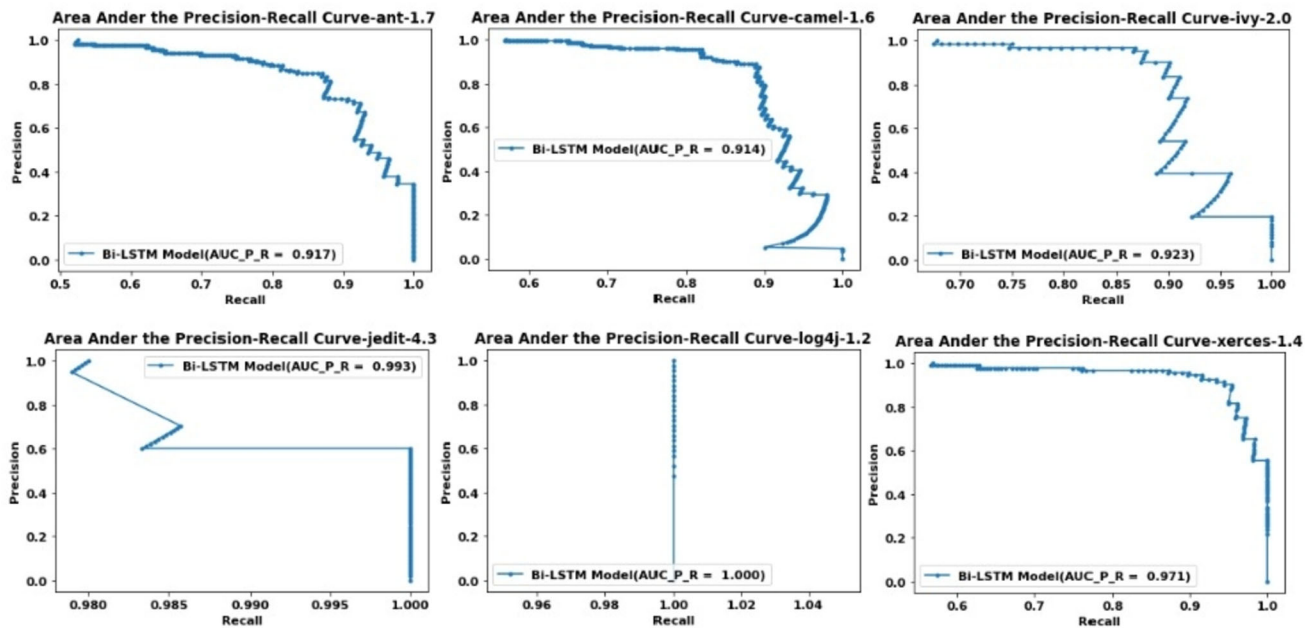


Fig. 16 AUCPR for the balanced datasets—SMOTE

Figure 6 shows the accuracy values of the model on the balanced datasets (using Random Oversampling). From the Figure, the model learned 91% accuracy for the ant dataset, 91% accuracy for the camel dataset, 90% accuracy for the ivy dataset, 99% accuracy for the jedit dataset, 99% accuracy for the log4j dataset, and 95% accuracy for xerces dataset at the 100th epoch.

Figure 7 shows the accuracy values of the model on the balanced datasets (using SMOTE). From the Figure, the model learned 84% accuracy for the ant dataset, 87% accuracy for the camel dataset, 89% accuracy for the ivy dataset, 99% accuracy for the jedit dataset, 100% accuracy for the log4j dataset, and 93% accuracy for xerces dataset at the 100th epoch.

Figures 8, 9, and 10 show the training and validation loss of the model on the original and balanced datasets. The vertical axis presents the loss of the model, and the horizontal axis illustrates the number of epochs. The loss indicates how bad a model prediction was.

Figure 8 shows the loss values of the model on the original datasets. From the Figure, the model loss is 0.152 for the ant dataset, 0.146 for the camel dataset, 0.105 for the ivy dataset, 0.030 for the jedit dataset, 0.041 for the log4j dataset, and 0.075 for the xerces dataset at the 100th epoch.

Figure 9 shows the loss values of the model on the balanced datasets (using Random Oversampling). From the Figure, the model loss is 0.073 for the ant dataset, 0.082 for the camel dataset, 0.092 for the ivy dataset, 0.009 for the jedit dataset, 0.012 for the log4j dataset, and 0.049 for the xerces dataset at the 100th epoch.

Figure 10 shows the loss values of the model on the balanced datasets (using SMOTE). From the Figure, the model loss is 0.124 for the ant dataset, 0.113 for the camel dataset, 0.101 for the ivy dataset, 0.011 for the jedit dataset, 0.001 for the log4j dataset, and 0.067 for the xerces dataset at the 100th epoch.

As shown in the figures, the accuracy of training and validation increases, and the loss decreases with increasing epochs. Regarding the high accuracy and low loss obtained by the proposed model, we note that the model is well-trained and validated.

Figures 11, 12, 13 below show the ROC curves of the model on the original and balanced datasets. The vertical axis presents the actual positive rate of the model, and the horizontal axis illustrates the false positive rate. The AUC is a sign of the performance of the model. The larger AUC is, the better the model performance will be. Based on the Figures, the values are encouraging and indicate our proposed model efficiency in SDP. The best AUC obtained by the proposed model in the original data sets is 94% on the xerces data set. The worst AUC is 60% on the log4j data set. The best AUC obtained by the proposed model in the balanced data sets (using random oversampling) is 99% on the jedit and log4j data sets, while the worst AUC is 93% on the camel and ivy data sets. The best AUC obtained by the proposed model in the balanced data sets (using SMOTE) is 100% on the log4j data set, while the worst AUC is 90% on the ant data set.

Figures 14, 15, 16 below show the AUCPR of the model on the original and balanced datasets. The vertical axis presents the precision of the model, and the horizontal axis

illustrates the recall. According to the figures, the best AUCPR obtained by the proposed model in the original data sets is 98% on the xerces data set. The worst AUCPR is 29% on the jedit data set. The best AUCPR obtained by the proposed model in the balanced data sets (using random oversampling) is 99% on the jedit and log4j data sets, while the worst AUCPR is 86% on the ivy data set. The best AUCPR obtained by the proposed model in the balanced data sets (using SMOTE) is 100% on the log4j data set, while the worst AUCPR is 91% on the ant and camel data sets.

To answer the research question—RQ2, we compared the results produced using our model with the results obtained using the baseline model (RF) based on six performance measures: accuracy precision, Recall, f-Measure, MCC, and AUC. Table 9 summarizes the comparison between Bi-LSTM and the baseline model (RF). According to Table 9, our model outperforms the baseline model in some datasets. We also compared the results produced using our model with those obtained in previous studies based on six performance measures: accuracy precision, recall, f-measure, MCC, and AUC. Table 10 compares the values of performance measures obtained by our Bi-LSTM Network and the performance values in previous studies.

The best values are indicated with bold text and “-” to indicate the approaches that did not provide results in a particular data set. According to Table 10, some of the results in the previous studies are better than ours. Still, in most cases, our model outperforms the other state-of-the-art approaches and provides better predictive performance.

7 The implication of the findings

The results have implications for researchers and practitioners. They are interested in quantitatively understanding the effectiveness and efficiency of applying data balancing methods with ML techniques in SDP. Furthermore, the formers are concerned about the qualitative perspective of the results. To summarize the main findings of our results and research questions, we provide implications related to effectiveness, efficiency, comparison, and relation with previous work, as follows:

Concerning RQ1, we observe from Tables 5, 6, 7, 8 and Figs. 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 that the model got good scores on the balanced datasets, and the results improved further due to balancing, which indicated that the proposed model performed well, and data balancing

Table 9 Performance measures of the baseline model (RF) and Bi-LSTM

Models	Datasets	Performance Measures					
		Accuracy	Precision	Recall	F-Measure	MCC	AUC
RF	ant	0.81	0.53	0.53	0.53	0.41	0.70
	camel	0.81	0.47	0.19	0.27	0.20	0.57
	ivy	0.89	0.57	0.44	0.50	0.44	0.69
	jedit	0.97	0.00	0.00	0.00	0.00	0.50
	log4j	0.98	0.97	1.00	0.99	0.69	0.75
	xerces	0.95	0.96	0.98	0.97	0.86	0.92
Averages		0.90	0.58	0.52	0.54	0.43	0.58
Bi-LSTM with Random Oversampling Technique	ant	0.91	0.89	0.94	0.91	0.82	0.95
	camel	0.91	0.87	0.98	0.92	0.81	0.93
	ivy	0.90	0.82	1.00	0.90	0.81	0.93
	jedit	0.99	0.98	1.00	0.99	0.97	0.99
	log4j	0.99	1.00	0.98	0.99	0.97	0.99
	xerces	0.95	0.98	0.92	0.95	0.89	0.97
Averages		0.94	0.92	0.97	0.94	0.87	0.96
Bi-LSTM with SMOTE Technique	ant	0.84	0.81	0.88	0.85	0.67	0.90
	camel	0.87	0.89	0.88	0.89	0.74	0.91
	ivy	0.89	0.83	0.97	0.89	0.78	0.94
	jedit	0.99	0.98	1.00	0.99	0.97	0.99
	log4j	1.00	1.00	1.00	1.00	1.00	1.00
	xerces	0.93	0.93	0.92	0.93	0.85	0.96
Averages		0.92	0.90	0.94	0.92	0.83	0.95

Table 10 Comparison of the proposed Bi-LSTM with other existing approaches

Approaches	Datasets	Performance measures					
		Accuracy	Precision	Recall	F-Measure	MCC	AUC
CNN [5]	ant, camel, ivy, jedit, log4j	–	–	–	0.39, 0.52, 0.31, 0.00, 0.97	0.30, 0.42, 0.25, 0.00, 0.00	–
Hybrid Neural Network model [6]	JEdit, IVY, Ant, Camel	0.97, 0.88, 0.81, 0.81	1.00 , 0.99, 0.93, 1.00	1.00 , 0.88, 0.84, 0.81	0.98, 0.93, 0.88, 0.89	–	–
HyGRAR [9]	JEdit, Ant	0.98, 0.96	0.70, 0.98	0.63, 0.85	–	–	0.81, 0.92
CNN [10]	ant, camel, ivy, jedit, log4j, xerces	0.85, 0.84, 0.95, 0.97, 0.97, 0.95	0.87, 0.81, 0.92, 0.94, 0.98, 0.93	0.82, 0.90, 0.98, 1.00 , 0.98, 0.98	0.85, 0.85, 0.95, 0.97, 0.98, 0.95	0.69, 0.69, 0.90, 0.93, 0.94, 0.90	0.91, 0.90, 0.98, 0.96, 0.99, 0.98
GRU [10]	ant, camel, ivy, jedit, log4j, xerces	0.83, 0.82, 0.95, 0.99, 0.96, 0.93	0.88, 0.82, 0.95, 0.98, 0.98, 0.92	0.81, 0.82, 0.95, 1.00 , 0.95, 0.94	0.85, 0.82, 0.95, 0.99, 0.96, 0.93	0.67, 0.63, 0.90, 0.97, 0.91, 0.85	0.89, 0.87, 0.98, 1.00 , 0.98, 0.97
LSTM [15]	Camel, Jedit, Log4j, Xerces	–	–	–	0.37, 0.44, 0.52, 0.26	–	–
LSTM [16]	Unified bug dataset (class-level, file-level)	0.93, 0.88	0.95, 0.94	0.92, 0.81	0.93, 0.87	0.86, 0.76	0.97, 0.93
GRU [16]	Unified bug dataset (class-level, file-level)	0.93, 0.88	0.94, 0.94	0.92, 0.81	0.93, 0.87	0.86, 0.76	0.96, 0.93
LSTM [18]	Bug report datasets (Eclipse platform UI and JDT)	0.67, 0.76	0.70, 0.76	0.86, 1.00	0.77, 0.86	–	–
CBIL [19]	Camel, JEdit, Xerces	–	–	–	0.93, 0.85, 0.95	–	0.96, 0.91, 0.98
SPFCNN [21]	CM1, JM1, KC1, PC1, MW1	–	–	–	–	0.85, 0.74, 0.78, 0.87, 0.80	0.92, 0.87, 0.88, 0.93, 0.90
BPDET [28]	CM1, JM1, KC1, MC1, PC1, MW1	–	–	–	0.84, 0.76, 0.83, 0.96, 0.92, 0.90	0.42, 0.23, 0.33, 0.14, 0.38, 0.33	0.75, 0.75, 0.81, 0.85, 0.88, 0.77
DP-ARNN [29]	Camel, Xerces, JEdit	–	–	–	0.51, 0.27, 0.56	–	0.79, 0.76, 0.82
LR [30]	Ant, Camel, IVY	–	–	–	0.52, 0.34, 0.30	–	–
K-NN [30]	Ant, Camel, IVY	–	–	–	0.53, 0.37, 0.30	–	–
MLP [30]	Ant, Camel, IVY	–	–	–	0.50, 0.38, 0.25	–	–
SVM [30]	Ant, Camel, IVY	–	–	–	0.50, 0.084, 0.28	–	–
LSTM [34]	Camel		0.51	0.41	0.46	–	–
Our model (Bi-LSTM with Random Oversampling Technique)	ant, camel, ivy, jedit, log4j, xerces	0.91, 0.91, 0.90, 0.99, 0.99, 0.95	0.89, 0.87, 0.82, 0.98, 1.00 , 0.98	0.94, 0.98, 1.00 , 1.00 , 0.98, 0.92	0.91, 0.92, 0.90, 0.99, 0.99, 0.95	0.82, 0.81, 0.81, 0.97, 0.97, 0.89	0.95, 0.93, 0.93, 0.99, 0.99, 0.97
Our model (Bi-LSTM with SMOTE Technique)	ant, camel, ivy, jedit, log4j, xerces	0.84, 0.87, 0.89, 0.99, 1.00 , 0.93	0.81, 0.89, 0.83, 0.98, 1.00 , 0.93	0.88, 0.88, 0.97, 1.00 , 1.00 , 0.92	0.85, 0.89, 0.89, 0.99, 1.00 , 0.93	0.67, 0.74, 0.78, 0.97, 1.00 , 0.85	0.90, 0.91, 0.94, 0.99, 1.00 , 0.96

Bold values indicate the best values of the performance measures

techniques play an essential role in improving the accuracy of Bi-LSTM Network in SDP.

Regarding RQ2, we observe from Table 10 that our proposed model achieved better performance than most existing approaches in terms of Accuracy, Precision, Recall, F-Measure, MCC, and AUC, which indicates that our model outperforms the other state-of-the-art approaches in SDP.

In summary, our study aimed to investigate the impact of data balancing techniques on the accuracy of the Bi-LSTM model in the domain of SDP and evaluate whether the proposed approach outperforms compared to state-of-the-art approaches for SDP. We conducted a comprehensive experiment using an imbalanced SDP dataset, employing techniques like random oversampling and SMOTE to address class imbalance in the training data. Our findings revealed that, in most cases across all datasets, the Bi-LSTM model trained with random oversampling and SMOTE techniques consistently outperformed the model trained without data balancing techniques (original datasets). The average values for Accuracy, Precision, Recall, F-Measure, MCC, AUC, and AUCPR exhibited significant improvements, while the average MSE decreased when employing random oversampling and SMOTE techniques, in contrast to the original datasets. These results suggest that oversampling is a suitable technique for addressing the class imbalance problem in SDP and the proposed model improves the existing works in SDP. It is important to note that every model has its strengths and limitations, and the choice of the most suitable model may depend on specific application requirements and datasets. Therefore, while the Bi-LSTM model shows promise, further research and experimentation are recommended to validate its superiority and to explore potential refinements. Our study contributes valuable insights to the field of SDP and provides a foundation for future investigations into the effectiveness of the Bi-LSTM model and other advanced approaches.

8 Threats to validity

This section discusses the threats to our study's validity and experiment limitations and how we mitigate them. It is vital to assess the threats to validity, such as construct, internal, external, and experiment limitations, particularly constraints on the search process and deviations from the standard practice.

Construct validity concerns the study's design and its possibility to reflect the actual goal of the research. To

avoid threats in study design, we have applied a procedure of systematic literature review. To ensure that the researched area is relevant to the study goal, we have cross-checked the research questions and adjusted them several times to address the business needs. Besides, the metrics considered may be a threat to our study. We only adopt static code metrics to predict defects. Thus, we cannot claim that we could generalize our conclusion to other metrics. However, many previous studies also widely adopted static code metrics [17, 33]. Another threat is the construction of ML models. We considered several aspects that could have influenced the study, i.e., data pre-processing, which features to consider, how to train the models, etc. However, the procedures followed in this respect are precise enough to ensure the study's validity.

Threats to internal validity are related to the correctness of the experiment's outcome or the study's process. The main threat to internal validity is datasets. The reference datasets are imbalanced datasets that show a lack in the actual distribution of the percentage of defects and non-defective classes. We manage this threat by modifying the original datasets to increase the data's realism regarding the defect's actual presence in the software system. The distribution of the dataset is modified by applying two data sampling techniques. Another threat is that most of our datasets have a small number of defects. These small number of defects make it challenging to generate statistically significant results; we tried to minimize that threat by applying standard performance measures for SDP. However, we acknowledge that several statistical tests [45] can be used to verify the statistical significance of our conclusions, which we plan to do in our future work.

External validity concerns relate to the generalization of our results. We tried to select and gather different datasets from different projects of the PROMISE repository to test our experiment. Our criteria in project selection were based on the ratio of defects. So, we chose projects with a high and low percentage of defects (projects with unbalanced classes) to help us apply data balancing techniques. We built our model to adapt the combination of RNNs with balancing techniques in SDP. We selected six open-source Java projects of the PROMISE dataset as our evaluation datasets. However, we cannot declare that our results can be generalized. Future replications of this study are necessary to confirm the generalizability of our findings.

The limitations of the experiments are summarized as follows. First, the datasets used in our experiments are limited to only six open-source Java projects. Second, our findings may not be enough to generalize to all software in the industrial domain.

9 Conclusion

Prediction of software defects plays a significant role in software quality assurance and helps software maintenance to run smoothly. Software defects can be predicted using ML-based static code analysis during development. Utilizing ML models for early defect prediction can significantly contribute to the reliability of software products. We suggested a novel approach, combining a Bi-LSTM network with oversampling techniques, to enhance the current state-of-the-art approaches for SDP. The implementation of oversampling techniques specifically aimed to address the issue of class imbalance, thereby improving the overall effectiveness of our approach in predicting software defects. To evaluate the effectiveness of the proposed Bi-LSTM network, we performed a series of experiments on six public software defect datasets. The results were compared with random forest (RF) as a baseline model and with existing SDP approaches. After analyzing the test results with a focus on Accuracy and F-measures, we have found that the proposed Bi-LSTM model demonstrates varying degrees of success across different datasets (ant, camel, ivy, jedit, log4j, and xerces). Specifically, it achieves accuracy rates of 91%, 91%, 90%, 99%, 99%, and 95%, respectively, when employing random oversampling techniques. When utilizing the SMOTE technique, the model attains accuracy rates of 84%, 87%, 89%, 99%, 100%, and 93% for the same datasets. Similarly, we have found that the proposed Bi-LSTM model demonstrates consistent performance across all datasets (ant, camel, ivy, jedit, log4j, and xerces). When employing the random oversampling technique, it yields F-measures of 91%, 92%, 90%, 99%, 99%, and 95%, respectively. Alternatively, with the utilization of the SMOTE technique, the model achieves F-measures of 85%, 89%, 89%, 99%, 100%, and 93%, respectively. Across the various datasets, random oversampling consistently outperformed the SMOTE technique, achieving an average accuracy of 94% and an average F-measure of 94%, while SMOTE yielded an average accuracy of 92% and an average F-measure of 92%. Consequently, we have concluded that random oversampling is the more appropriate technique for mitigating the class imbalance issue in SDP. In comparison with the baseline model (RF), we found that the proposed Bi-LSTM model uses random oversampling and SMOTE techniques with an average accuracy of 94 and 92%, respectively compared with RF (90%). Our results showed that the proposed Bi-LSTM network with random oversampling and SMOTE techniques improves the average accuracy by 4 and 2%, respectively, compared to RF. Likewise, the proposed Bi-LSTM model using random oversampling and SMOTE techniques with an average

f-measure of 94 and 92%, respectively compared with RF (54%). Our results showed that the proposed Bi-LSTM network with random oversampling and SMOTE techniques improves the average f-measure by 40 and 38%, respectively, compared to RF. The proposed Bi-LSTM model outperforms existing state-of-the-art SDP approaches significantly and substantially in most cases in terms of accuracy, precision, recall, f-measure, MCC, and AUC. The experimental findings showcase the superior performance of the proposed approach, highlighting the positive impacts of integrating Bi-LSTM with oversampling techniques. Our approach stands out as a more promising solution for tackling class imbalance in SDP when compared to previous methods. This research has significant implications for software developers and practitioners who aim to improve software quality and reduce the risk of defects in software systems. Experimental results demonstrate the robustness of the proposed approach, ensuring its ability to maintain high predictive performance across different SDP scenarios. In addition, the robustness and accuracy of the proposed approach will be evaluated on different and large datasets (e.g., Prop dataset) in our future work.

Acknowledgements The authors gratefully acknowledge the financial assistance from the Institute of Information Science, Faculty of Mechanical Engineering and Informatics, University of Miskolc.

Author contributions NAAK conducted the experiments, performed the data analyses, wrote the manuscript text, and prepared figures and tabular data. KN reviewed the experiments, the data analyses, manuscript text, and figures and tabular data. Both authors, discussed the results, and contributed to the final manuscript. The authors read and approved the final manuscript.

Funding Open access funding provided by University of Miskolc.

Data availability The datasets for the current study are available from <https://www.kaggle.com/datasets/nazgolnikravesh/software-defect-prediction-dataset>.

Declarations

Competing interests The authors declared that they have no competing interests in this work. We declare that we do not have any commercial or associative interest that represents a conflict of interest in connection with the work submitted.

Ethics approval Not applicable.

Consent to participate Not applicable.

Consent for publication Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate

if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Li, Z., Jing, X.Y., Zhu, X.: Progress on approaches to software defect prediction. *IET Softw.* **12**(3), 161–175 (2018). <https://doi.org/10.1049/iet-sen.2017.0148>
2. Ayon, S.I.: Neural network based software defect prediction using genetic algorithm and particle swarm optimization. In: 1st International conference on advances in science, engineering and robotics technology, Dhaka, Bangladesh. (2019). <https://doi.org/10.1109/ICASERT.2019.8934642>
3. Mustaqeem, M., Saqib, M.: Principal component based support vector machine (PC-SVM): a hybrid technique for software defect detection. *Clust. Comput.* **24**, 2581–2595 (2021). <https://doi.org/10.1007/s10586-021-03282-8>
4. Tong, H., Liu, B., Wang, S.: Software defect prediction using stacked denoising autoencoders and two-stage ensemble learning. *Inf. Softw. Technol.* **96**, 94–111 (2018). <https://doi.org/10.1016/j.infsof.2017.11.008>
5. Pan, C., Lu, M., Xu, B., et al.: An improved CNN model for within-project software defect prediction. *Appl. Sci.* **9**(10), 2138 (2019). <https://doi.org/10.3390/app9102138>
6. Kumar, R.S., Sathyanarayana, B.: Adaptive genetic algorithm based artificial neural network for software defect prediction. *Glob. J. Comput. Sci. Technol.* **15**(1), 23–32 (2015)
7. Manjula, C., Florence, L.: Deep neural network based hybrid approach for software defect prediction using software metrics. *Clust. Comput.* **22**(4), 9847–9863 (2019). <https://doi.org/10.1007/s10586-018-1696-z>
8. Anbu, M., Anandha Mala, G.S.: Feature selection using firefly algorithm in software defect prediction. *Clust. Comput.* **22**(5), 10925–10934 (2019). <https://doi.org/10.1007/s10586-017-1235-3>
9. Miholca, D.L., Czibula, G., Czibula, I.G.: A novel approach for software defect prediction through hybridizing gradual relational association rules with artificial neural networks. *Inf. Sci.* **441**, 152–170 (2018). <https://doi.org/10.1016/j.ins.2018.02.027>
10. Khleel, N.A.A., Nehéz, K.: A novel approach for software defect prediction using CNN and GRU based on SMOTE Tomek method. *J. Intell. Inf. Syst.* **60**(3), 673–707 (2023). <https://doi.org/10.1007/s10844-023-00793-1>
11. Akour, M., Melhem, W.Y.: Software defect prediction using genetic programming and neural networks. *Int. J. Open Sour. Softw. Process.* **8**(4), 32–51 (2017). <https://doi.org/10.4018/IJOSSP.2017100102>
12. Khleel, N.A.A., Nehéz, K.: A new approach to software defect prediction based on convolutional neural network and bidirectional long short-term memory. *Prod. Syst. Inf. Eng.* **10**(3), 1–18 (2022). <https://doi.org/10.32968/psaie.2022.3.1>
13. Arar, Ö.F., Ayan, K.: Software defect prediction using cost-sensitive neural network. *Appl. Soft Comput.* **33**, 263–277 (2015). <https://doi.org/10.1016/j.asoc.2015.04.045>
14. Jayanthi, R., Florence, L.: Software defect prediction techniques using metrics based on neural network classifier. *Clust. Comput.* **22**(1), 77–88 (2019). <https://doi.org/10.1007/s10586-018-1730-1>
15. Deng, J., Lu, L., Qiu, S.: Software defect prediction via LSTM. *IET Softw.* **14**(4), 443–450 (2020). <https://doi.org/10.1049/iet-sen.2019.0149>
16. Khleel, N.A.A., Nehéz, K.: Improving the accuracy of recurrent neural networks models in predicting software bug based on undersampling methods. *Indones. J. Electr. Eng. Comput. Sci.* **32**(1), 478–493 (2023). <https://doi.org/10.11591/ijeecs.v32.i1.pp478-493>
17. Chen, L., Fang, B., Shang, Z., Tang, Y.: Negative samples reduction in cross-company software defects prediction. *Inf. Softw. Technol.* **62**, 67–77 (2015). <https://doi.org/10.1016/j.infsof.2015.01.014>
18. Ye, X., Fang, F., Wu, J., Bunescu, R., Liu, C.: December. Bug Report Classification using LSTM architecture for more accurate software defect locating. In: International conference on machine learning and applications, Orlando, FL, U.S.A., pp. 1438–1445. (2018). <https://doi.org/10.1109/ICMLA.2018.00234>
19. Farid, A.B., Fathy, E.M., Eldin, A.S., Abd-Elmegid, L.A.: Software defect prediction using hybrid model (CBIL) of convolutional neural network (CNN) and bidirectional long short-term memory (Bi-LSTM). *PeerJ Comput. Sci.* **7**, e739 (2021). <https://doi.org/10.7717/peerj-cs.739>
20. Zhou, X., Lu, L.: Defect prediction via LSTM based on sequence and tree structure. In: IEEE 20th international conference on software quality, reliability and security, Macau, China, pp. 366–373. (2020). <https://doi.org/10.1109/QRS51102.2020.00055>
21. Zhao, L., Shang, Z., Zhao, L., Zhang, T., Tang, Y.Y.: Software defect prediction via cost-sensitive Siamese parallel fully-connected neural networks. *Neurocomputing* **352**, 64–74 (2019). <https://doi.org/10.1016/j.neucom.2019.03.076>
22. Öztürk, M.M.: Which type of metrics are useful to deal with class imbalance in software defect prediction? *Inf. Softw. Technol.* **92**, 17–29 (2017). <https://doi.org/10.1016/j.infsof.2017.07.004>
23. Ali, M.M., Huda, S., Abawajy, J., et al.: A parallel framework for software defect detection and metric selection on cloud computing. *Clust. Comput.* **20**, 2267–2281 (2017). <https://doi.org/10.1007/s10586-017-0892-6>
24. Mohammed, B., Awan, I., Ugail, H., et al.: Failure prediction using machine learning in a virtualised HPC system and application. *Clust. Comput.* **22**, 471–485 (2019). <https://doi.org/10.1007/s10586-019-02917-1>
25. Samir, M., El-Ramly, M., Kamel, A.: Investigating the use of deep neural networks for software defect prediction. In: IEEE/ACS 16th international conference on computer systems and applications, Abu Dhabi, United Arab, pp. 1–6. (2019). <https://doi.org/10.1109/AICCSA47632.2019.9035240>
26. Alsaedi, A., Khan, M.Z.: Software defect prediction using supervised machine learning and ensemble techniques: a comparative study. *J. Softw. Eng. Appl.* **12**(5), 85–100 (2019). <https://doi.org/10.4236/jsea.2019.125007>
27. Dam, H.K., Pham, T., Ng, S.W., Tran, T., Grundy, J., Ghose, A., Kim, T., Kim, C.J.: A deep tree-based model for software defect prediction. *arXiv* (2018). <https://doi.org/10.48550/arXiv.1802.00921>
28. Pandey, S.K., Mishra, R.B., Tripathi, A.K.: BPDET: an effective software bug prediction model using deep representation and ensemble learning techniques. *Expert Syst. Appl.* **144**, 113085 (2020). <https://doi.org/10.1016/j.eswa.2019.113085>
29. Fan, G., Diao, X., Yu, H., Yang, K., Chen, L.: Software defect prediction via attention-based recurrent neural network. *Sci. Programm.* (2019). <https://doi.org/10.1155/2019/6230953>
30. Khuat, T.T., Le, M.H.: Evaluation of sampling-based ensembles of classifiers on imbalanced data for software defect prediction problems. *SN Comput. Sci.* **1**(2), 1–16 (2020). <https://doi.org/10.1007/s42979-020-0119-4>

31. Majd, A., Vahidi-Asl, M., Khalilian, A., Poorsarvi-Tehrani, P., Haghighi, H.: SLDeep: statement-level software defect prediction using deep-learning model on static code features. *Expert Syst. Appl.* **147**, 113156 (2020). <https://doi.org/10.1016/j.eswa.2019.113156>
32. Bani-Salameh, H., Sallam, M.: A deep-learning-based bug priority prediction using RNN-LSTM neural networks. *E-Inf. Softw. Eng. J.* **15**(1), 29–45 (2021). <https://doi.org/10.37190/e-Inf210102>
33. Feng, S., Keung, J., Yu, X., Xiao, Y., Zhang, M.: Investigation on the stability of SMOTE-based oversampling techniques in software defect prediction. *Inf. Softw. Technol.* **139**, 106662 (2021). <https://doi.org/10.1016/j.infsof.2021.106662>
34. Liang, H., Yu, Y., Jiang, L., Xie, Z.: Seml: a semantic LSTM model for software defect prediction. *IEEE Access* **7**, 83812–83824 (2019). <https://doi.org/10.1109/ACCESS.2019.2925313>
35. Yang, Z., Qian, H.: Automated parameter tuning of artificial neural networks for software defect prediction. In: *Proceedings of the 2nd international conference on advances in image processing*, New York, NY, United States, pp.203–209. (2018). <https://doi.org/10.1145/3239576.3239622>
36. Verma, Y.: Complete Guide to Bidirectional LSTM (with Python Codes). *Analytics India Magazine*. analyticsindiamag.com/complete-guide-to-bidirectional-lstm-with-python-codes/. Accessed 20 Nov 2021
37. Nikravesh, N.: Software Defect Prediction Dataset. Kaggle. www.kaggle.com/datasets/nazgolnikravesh/software-defect-prediction-dataset. Accessed 3 Aug 2021
38. Xia, X., Lo, D., Pan, S.J., Nagappan, N., Wang, X.: Hydra: massively compositional model for cross-project defect prediction. *IEEE Trans. Softw. Eng.* **42**(10), 977–998 (2016). <https://doi.org/10.1109/TSE.2016.2543218>
39. Khleel, N.A.A., Nehéz, K.: Comprehensive study on machine learning techniques for software bug prediction. *Int. J. Adv. Comput. Sci. Appl.* **12**(8), 726–735 (2021). <https://doi.org/10.14569/IJACSA.2021.0120884>
40. Shippey, T., Bowes, D., Hall, T.: Automatically identifying code features for software defect prediction: using A.S.T. N-grams. *Inf. Softw. Technol.* **106**, 142–160 (2019). <https://doi.org/10.1016/j.infsof.2018.10.001>
41. Khan, M.Z.: Hybrid ensemble learning technique for software defect prediction. *Int. J. Modern Educ. Comput. Sci.* **12**(1), 1–10 (2020). <https://doi.org/10.5815/ijmecs.2020.01.01>
42. Aquil, M.A.I., Ishak, W.H.W.: Predicting software defects using machine learning techniques. *Int. J. Adv. Trends Comput. Sci. Eng.* **9**(4), 6609–6616 (2020). <https://doi.org/10.30534/ijatcse/2020/352942020>
43. Jain, S., Saha, A.: Improving performance with hybrid feature selection and ensemble machine learning techniques for code smell detection. *Sci. Comput. Program.* **212**, 102713 (2021). <https://doi.org/10.1016/j.scico.2021.102713>
44. Bashir, K., Li, T., Yohannese, C.W.: An empirical study for enhanced software defect prediction using a learning-based framework. *Int. J. Comput. Intell. Syst.* **12**(1), 282–298 (2018). <https://doi.org/10.2991/ijcis.2018.125905638>
45. Arcuri, A., Briand, L.: A hitchhiker’s guide to statistical tests for assessing randomized algorithms in software engineering. *Softw. Test. Verif. Reliab.* **24**(3), 219–250 (2014). <https://doi.org/10.1002/stvr.1486>

Publisher’s Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Nasraldeen Alnor Adam Khleel received his B.Sc. degree in Information Systems from the University of Kassala, Kassala-Sudan, in 2011. He got an M.Sc. degree in Software Engineering at Khartoum University, Khartoum-Sudan, in 2015. He is currently pursuing a PhD at the University of Miskolc under the Faculty of Mechanical Engineering and Informatics, Miskolc-Hungary, since 2019. His primary research interests include software engineering, machine learning and artificial intelligence. He can be contacted by email: nasr.alnor@uni-miskolc.hu.



Károly Nehéz received his M.Sc. degree in mechanical engineering from the University of Miskolc, Hungary, in 1997 and a Ph.D. in Software Engineering in 2003. He currently works as an associate professor at the Institute of Computer Science, head of the institute since 2019. His primary research interest is software engineering, although he has concurrent research in machine learning and artificial intelligence. He can be contacted by email: karoly.nehez@uni-miskolc.hu.